DOS ARQUIVOS-FONTE À RECONSTRUÇÃO FIEL

CARONINHAS PARTY

Documentação técnica, arquitetura de infraestrutura, especificação funcional de três jogos, design system visual e manual para reconstrução por inteligência artificial.

PORTAL WEB

HTML · CSS · JavaScript · Express · Socket.IO · PostgreSQL

RONIZADA QUEST

API V1.3 · Banco trivia · Perguntas e prazo autoritativo

TRÊS MODALIDADES

Impostor · Quem Sou Eu? · Corrida do Conhecimento

REFERÊNCIA VISUAL

Rosa e creme · Chakra Petch + Nunito · Janelas retrô

Versão documentada: V20.20.4
Fonte: código e testes dos pacotes de referência · Outubro de 2026
Segredos, dados de usuários e banco de produção não incluídos

DOCUMENTAÇÃO COMPLETA — CARONINHAS PARTY
CARONINHAS PARTY — DOCUMENTAÇÃO MESTRA
Especificação técnica, de produto e visual, orientada à reconstrução
1. Como ler e como entregar este material a outra IA
2. O que é o produto
3. Princípios de identidade e consistência
4. Distinção entre versão, pacote e deploy
5. O que este conjunto NÃO contém
6. Condições de uma reconstrução fiel
01 — ARQUITETURA, STACK, INFRAESTRUTURA E DEPLOY
1. Visão de contexto
2. Tecnologias confirmadas pelo código
3. Estrutura do portal e responsabilidades
4. Estrutura da Ronizada Quest
5. Modelo de hospedagem
6. Inicialização do backend
7. Estado em memória e consequência operacional
8. Dependências externas e risco
9. Runtime/deploy recomendado para reprodução
02 — MODELAGEM DE DADOS, SQL, ESTADOS E PERSISTÊNCIA
1. Separação lógica
2. Modelo principal (`database/schema.sql` + migrações)
3. Detalhes de modelagem importantes
4. Schema `trivia` — implementação V1.3
5. Isolamento e permissões
6. Estado não persistido
7. Backups e mudança segura
8. SQL de auditoria seguro (leitura)
03 — CONTRATOS DE INTEGRAÇÃO: REST + SOCKET.IO
1. Convenções
2. Rotas HTTP do portal
3. Formato público de sala (`publicRoom`)
4. Socket.IO — comandos do cliente para servidor
5. Socket.IO — mensagens servidor para cliente
6. Reconexão e segurança da admissão
7. Corrida — estado Socket.IO (`trivia-state`)
8. Erros que outra IA precisa evitar
04 — ESPECIFICAÇÃO DE PRODUTO, REGRAS E MÁQUINAS DE ESTADOS
1. Padrão transversal a todas as modalidades
2. Impostor
3. Quem Sou Eu?
4. Corrida do Conhecimento — especificação normativa da V20.20.4
5. Regras de reaproveitamento obrigatório
05 — FRONTEND, NAVEGAÇÃO, DOM, MODAIS, ESTADOS E RESPONSIVIDADE
1. Modelo de frontend: SPA sem framework
2. Inventário das telas principais
3. Estado de identidade do cliente

4. Componentes reutilizáveis de interface
5. Página de criação da Corrida
6. Lobby e corrida em andamento
7. Tela de campeão
8. Telas de Impostor / Quem Sou Eu? — o que replicar
9. Acessibilidade e experiência real
10. Fluxo de telas ao implementar nova modalidade

06 — DESIGN SYSTEM VISUAL: IDENTIDADE, TIPOGRAFIA, CORES, MEDIDAS E GRID
1. Linguagem de interface
2. Tokens CSS globais (valores reais de `public/style.css`)
3. Tipografia exata da V20.20.4
4. Estrutura dimensional da janela
5. Componentes — geometria, hierarquia e interação
6. Sistema de espaçamento e proporção
7. Especificidade, cascata e regressão
8. Responsive e viewport
9. Identidade por modalidade
10. Referências de UI e capturas

07 — RONIZADA QUEST V1.3 — SERVIÇO DE PERGUNTAS, PRAZOS E SORTEIO
1. Função e limites
2. Contratos REST exatos de alto nível
3. Criar rodada
4. Enviar resposta
5. Encerrar e pontuar
6. Comunicação Portal → Trivia
7. Categorias e regras da configuração
8. Revisão e origem do acervo
9. Deploy e preservação de dados
10. Casos de teste essenciais

08 — SEGURANÇA, CONTAS, AMBIENTE, OPERAÇÃO E INCIDENTES
1. Modelo de autenticação existente
2. Recuperação de senha
3. Fotos, avatares e privacidade
4. Salas privadas, host e Socket.IO
5. Backend para backend e gestão de segredos
6. Inicialização e migrações
7. Observabilidade realmente presente
8. Incidentes comuns — diagnóstico sem expor segredos
9. Rotina de publicação recomendada
10. Riscos técnicos e recomendações (NÃO implantadas automaticamente)

09 — QA, TESTES DE REGRESSÃO, ACEITAÇÃO E FIDELIDADE
1. Objetivo de qualidade
2. Matriz de validação obrigatória
3. Testes de segurança/privacidade
4. Testes com conexão real
5. Validação visual pixel comparativa
6. Checklist para liberação
7. Escopo não testado automaticamente

10 — PLAYBOOK DE RECONSTRUÇÃO PARA OUTRA INTELIGÊNCIA ARTIFICIAL
1. Missão
2. Não começar por CSS isolado
3. Matriz de fidelidade
4. Contratos de aceitação da réplica
5. Arquivos prioritários para leitura por agentes
6. Processo de implementação por sprints (sugestão, não estado atual)
7. Prompts para revisão especializados
8. Alterações proibidas sem autorização específica
9. Entrega requerida do agente construtor

11 — INVENTÁRIO GERADO DIRETAMENTE DOS ARQUIVOS-FONTE
1. Hash SHA-256 dos arquivos essenciais
2. Roteamento REST detectado por expressões `app.get/post/patch/put`
3. Eventos Socket.IO recebidos e emitidos (ocorrências detectadas)
4. Todas as seções de tela de `index.html`
5. Categorias no dataset de origem da Trivia (não refletindo liberação em banco)
6. Avatares preset e mídias de interface
7. Inventário de testes
8. CSS: tokens globais declarados no início do arquivo
9. Arquivos do projeto portal (sem datasets binários e segredos)
10. Arquivos da API (sem questões individuais e segredos)
11. Notas de rastreabilidade

15 — ATLAS VISUAL DE REFERÊNCIA
1. Tela de configuração em desenvolvimento — problema histórico
2. Layout de criação V20.20.4 — prévia em 1366 px
3. Layout de criação V20.20.4 — prévia móvel
4. Corrida em andamento — exemplo de gameplay real enviado pelo usuário
5. Vitória ainda exibida no jogo — problema histórico corrigido
6. Hierarquia visual esperada do produto

DOCUMENTAÇÃO COMPLETA — CARONINHAS PARTY

V20.20.4 · Manual técnico, de infraestrutura, visual e de reconstrução

CARONINHAS PARTY — DOCUMENTAÇÃO MESTRA

Especificação técnica, de produto e visual, orientada à reconstrução

Linha-base documental: pacote `CARONINHAS_PARTY_V20_20_4_REFINAMENTO_VISUAL.zip` e serviço `RONIZADA_QUEST_TRIVIA_API_V1_3_DISCLOUD.zip`, analisados em 24/10/2026.
Abrangência: site, frontend, backend, autenticação, PostgreSQL, salas multiplayer, Impostor, Quem Sou Eu?, Corrida do Conhecimento, API Trivia, estilos, infraestrutura, operação e critérios de réplica.
Estado: documentação da implementação de referência, **não** auditoria conclusiva do ambiente remoto. Comportamentos de servidor/infraestrutura que exigem medição em produção são assinalados como *verificação necessária*.
Idioma: português brasileiro. **Público-alvo:** desenvolvedor pleno/sênior, arquiteto de software, designer de interface, QA e agentes de codificação.

1. Como ler e como entregar este material a outra IA

Este pacote foi estruturado para permitir a **reconstrução funcional e visual** usando o código-fonte como fonte máxima de verdade. Leia os documentos na sequência: 01 Arquitetura, 02 Dados, 03 Contratos HTTP e sockets, 04 Jogos, 05 Frontend, 06 Design system, 07 Trivia, 08 Segurança, 09 Testes, 10 Reconstrução, 11 Inventário verificável. As capturas em `referencias-visuais/` complementam o CSS, mas **não** o substituem.

Ordem de precedência de fontes

1. Arquivo-fonte da mesma versão (JS, HTML, CSS, SQL) e seus testes.
2. Documentação versionada inclusa nos ZIPs.
3. Referências visuais da versão e telas reais disponíveis.
4. Especificação neste pacote (síntese interpretada do código).
5. Sugestões de melhoria — devem ser explicitamente separadas do que existe.

Regra fundamental: uma réplica não deve inventar novas modalidades, botões, efeitos, bibliotecas, rotas, banco, personagens ou fluxos; também não deve eliminar recursos por não serem parte da tela em foco. A meta é equivalência antes de modernização.

2. O que é o produto

Caroninhas Party é um portal web brasileiro de minijogos multiplayer em tempo real, com identidade visual retrô/desktop, biblioteca de jogos, acesso como visitante ou conta, perfis e avatares, salas por código, salas públicas/privadas, anfitrião, confirmação de presença, reconexão e espectadores. Modalidades ativas desta linha-base:

Slug	Modalidade	Núcleo da mecânica
`/impostor`	Impostor	Uma pessoa desconhece a palavra; discussão, votação, eliminações e vitória por lado
`/quem-sou-eu`	Quem Sou Eu?	Personagens sorteados ocultos para o próprio participante; revisão entre pares, palpites e classificações
`/corrida-do-conhecimento`	Corrida do Conhecimento	Pergunta compartilhada, resposta individual até o prazo, torre 0–10 e desempate

A rota legada `/quiz-e-caos` redireciona para `/`; não a trate como quarta modalidade ativa.

3. Princípios de identidade e consistência

- **Um único site e um único shell visual:** fundo rosa quadriculado, janelas retrô, barra de título com três pontos, bordas grossas escuras, sombra dura deslocada, superfícies creme, botões e rótulos comuns.
- **Biblioteca e jogo compartilham padrões:** cards de seleção, criação/entrada em sala, identidade do jogador, avatares, sala/lobby, status *pronto*, controles de anfitrião, espera, retorno, estados de erro e reconexão.
- **Cada jogo altera somente suas particularidades:** Impostor (palavra/voto), Quem Sou Eu? (personagens/revisão), Corrida (tags/pergunta/cronômetro/torres/desempate/campeão).
- **Não criar tema independente por jogo.** O design escuro originalmente utilizado numa beta da Corrida foi rejeitado; a V20.20.4 usa o tema geral do Caroninhas.
- **Não misturar credenciais:** a chave da Ronizada Quest reside no backend do site, nunca no navegador.

4. Distinção entre versão, pacote e deploy

O site apresenta a sequência visual/funcional **V20.20.4** nos arquivos/leituras; o `package.json` do site registra `version: 5.0.20`. Não normalize esses dois identificadores silenciosamente. A API separada registra `version: 1.3.0`. O `discloud.config` incluído no ZIP da V20.20.4 preserva **identificadores de aplicação de TESTE**; o endereço de produção `caroninhas-party.discloud.dev` depende de um manifesto de produção mantido pelo operador. Não copie o manifesto de teste para o aplicativo principal.

5. O que este conjunto NÃO contém

- Senhas, cookies de sessão, `DATABASE_URL` real, chaves de API, tokens IGDB/TMDB e banco em produção.
- Exportação integral das tabelas PostgreSQL de produção, de sessões ou de histórico.
- Dump das fotos de perfil armazenadas em `users.profile_image`.
- Garantia de equivalência pixel a pixel em todos os navegadores/resoluções ou da disponibilidade de provedores externos.
- Evidência de orquestrador, cache distribuído, CDN próprio, Docker/Kubernetes, pipeline CI/CD ou observabilidade avançada: **não presumir que existem**.

6. Condições de uma reconstrução fiel

A reconstrução precisa reproduzir, no mínimo, a navegação, fluxos de acesso, eventos do multiplayer, os três estados de jogo, banco de dados, progresso/reconexão, busca de imagens do Quem Sou Eu?, acervo e temporização da Trivia, configuração da Corrida por tags, estilo retrô com fontes corretas e responsividade. O acervo exato e as imagens originais requerem os dados/arquivos originais e políticas de licença; uma IA não os recriará fielmente apenas com descrição.

Glossário curto: *host/anfitrião* = dono lógico da sala; *playerToken* = identificador local de participação (não substitui sessão autenticada); *sessionSecret* = segredo por admissão na sala, usado na retomada; *round* = rodada; *phase* = estado do processo; *trivia* = API de perguntas e também gameType da Corrida; *finalista* = jogador que alcança 10 e disputa desempate.

---

01 — ARQUITETURA, STACK, INFRAESTRUTURA E DEPLOY

1. Visão de contexto

Arquitetura de **dois serviços Node.js + PostgreSQL**, com o navegador abrindo uma aplicação monolítica renderizada por HTML/CSS/JS puro e usando WebSockets via Socket.IO para partidas. O portal é o **único frontend**. Para a Corrida, seu backend consulta a **API de perguntas separada** pela rede; o navegador não se autentica diretamente na Trivia API.

```
flowchart LR
    B[Browser\nHTML + CSS + JavaScript] -->|HTTPS REST; cookie HttpOnly| W[Portal Caroninhas Party\nExpress + Node.js]
    B <-->|Socket.IO\nconexão bidirecional| W
    W -->|node-postgres / SQL| P[(PostgreSQL 17\nschema public)]
    W -->|HTTPS /api/v1\nx-api-key somente no servidor| T[Ronizada Quest V1.3\nExpress + Node.js]
    T -->|usuário restrito / SQL| Q[(mesmo PostgreSQL\nschema trivia)]
    W -->|HTTP APIs públicas autenticadas ou abertas| I[IGDB, TMDB, AniList, TVMaze, Wikipedia/Wikidata/Commons]
```

Observação: `P` e `Q` representam **schemas lógicos** do PostgreSQL informado no histórico do projeto; não são necessariamente servidores físicos diferentes. A topologia exata da Discloud deve ser verificada no painel da hospedagem.

2. Tecnologias confirmadas pelo código

Camada	Tecnologia	Arquivos evidência	Notas
Frontend	HTML5 sem framework SPA	`public/index.html`	Diversas `<section class="screen">`, visibilidade por classe `.hidden`
Estilo	CSS3 puro	`public/style.css`	Variáveis CSS, Flexbox, Grid, queries de responsividade, estados, animações
Cliente	JavaScript vanilla	`public/script.js`	Manipulação do DOM, estado em objetos JS, `localStorage`, `sessionStorage`, `fetch`, Socket.IO
Tempo real	Socket.IO 4.8.x	`server.js` + cliente	Salas, broadcast, callbacks, reconexão
HTTP	Express 5.2.x	`server.js`, `auth.js`	Static SPA, endpoints JSON e perfis
Runtime	Node.js	`package.json`	Portal necessita `fetch` e `AbortSignal.timeout` nativos: usar Node >= 20 por segurança operacional
Persistência	PostgreSQL + `pg` 8.13.x	`db.js`, SQL	Pools de até 10 conexões no portal
Ambiente	`dotenv` 16.4.x	`db.js`, app Trivia	Credenciais por variáveis de ambiente
API Trivia	Express 5.2.x + Node >=20	`trivia/src/*`	Resposta individual com carimbo temporal no banco
Criptografia	`node:crypto`	`auth.js`, `server.js`, `trivia-race.js`	scrypt, SHA-256, segredo aleatório, randomUUID, comparação segura
Testes	`node:test`	`tests/*.test.js`	Testes de regras, handlers, compatibilidade e checagem estática

Camada	Tecnologia	Arquivos evidência	Notas
Hospedagem	Discloud (arquivos `discloud.config`)	Ambos pacotes	Dois apps Node, conexão com PostgreSQL privado via VLAN quando exigida

Não há evidência de React, Vue, Angular, Vite, TypeScript, Tailwind, Next.js, Django ou gRPC neste site. Não introduzir frameworks numa réplica estrita, a menos que se aceite reescrever o produto mantendo seus contratos.

3. Estrutura do portal e responsabilidades

```
caroninhas-party/
├── server.js                      # Express, HTTP, Socket.IO, salas, Impostor, Quem Sou Eu?
├── trivia-race.js                 # Corrida: filtros, estados, REST de perguntas, pontuação
├── auth.js                        # Usuários, sessões, cookies, perfis, stats, preferências
├── db.js                          # Pool pg + query + testConnection
├── whoami-images.js               # Provedores externos de imagens e fallback
├── public/
│   ├── index.html                 # TODAS as telas, modais, templates e SVG sprite
│   ├── script.js                  # SPA, eventos, sockets, modais, timers, telas
│   ├── style.css                  # Design system e regras específicas de cada tela
│   ├── caroninhas-party-banner.png
│   └── avatars/                   # 10 animais PNG
├── database/
│   ├── schema.sql                 # schema principal
│   ├── seed.sql                   # categorias, níveis e conteúdo-semente
│   ├── migrations/001..005.sql    # contas, quiz legado, preferências, expansão do acervo
│   └── data/quiz_questions.jsonl  # acervo local legado (não confundir com Ronizada)
├── scripts/
│   ├── bootstrap-db.js
│   ├── init-db.js
│   ├── run-migrations.js
│   ├── load-quiz-questions.js
│   └── whoami-image-report.js
├── tests/                         # testes Node
├── package.json / package-lock.json
├── discloud.config                # IMPORTANTE: versão de teste no ZIP analisado
└── .env.example                   # placeholders; não tem credenciais reais
```

Principais decisões arquiteturais: **monólito pequeno para site + API dedicada para perguntas**; separação funcional do código da Corrida em `trivia-race.js`; provedores de imagem encapsulados em `whoami-images.js`; autenticação encapsulada em `auth.js`; nenhum bundler exigido para browser.

4. Estrutura da Ronizada Quest

```
ronizada-quest/
├── index.js                       # entrada principal
├── src/server.js                  # bootstrap HTTP, proteção de ambiente e encerramento
├── src/app.js                     # rotas JSON e operações SQL
├── src/db.js                      # pool PostgreSQL
├── src/bootstrap.js               # migração/importação condicionais
├── src/config.js                  # exigência de segredos
├── src/rounds.js                  # IDs, prazos e respostas
├── src/normalize.js               # alternativas/sorteio/normalização
├── src/validation.js              # filtro e validação de requisição
├── src/importer.js                # importação do JSONL
├── sql/001_init.sql               # trivia.questions, rounds, round_answers
├── sql/000_PREPARAR_SCHEMA_E_USUARIO.sql # preparação administrativa UMA VEZ
├── scripts/migrate.js, import.js, bootstrap.js
├── data/questions.jsonl           # acervo de perguntas PT-BR
├── tests/
```

```
├── discloud.config / .env.example
└── package.json
```

5. Modelo de hospedagem

- **Portal de produção observado no histórico:** `https://caroninhas-party.discloud.dev`.
- **API Trivia:** `https://ronizada-quest.discloud.dev`, com verificação `/health` retornando versão `1.3.0` em teste anterior.
- **Bancos:** ambos usam PostgreSQL; Trivia limitada ao schema `trivia` e usuário com privilégios restritos. Histórico cita host privado e VLAN. Os detalhes de credenciais e host privado não estão reproduzidos deliberadamente.
- **Portal:** `discloud.config` do ZIP analisado usa `TYPE=site`, `MAIN=server.js`, `START=npm start`, `RAM=512`, `AUTORESTART=true`, `VLAN=true`, mas o **ID/hostname são de teste**. A configuração de produção precisa ser obtida da aplicação original, não inferida daqui.
- **API:** também usa app Discloud distinto `ronizada-quest`, com VLAN necessária ao acesso ao banco privado no cenário histórico.
- **Exposição externa:** HTTP/HTTPS pela plataforma; cliente se conecta ao Socket.IO no mesmo host da aplicação principal. Portas locais padrão `8080` para site/API e `5432` para PostgreSQL.

6. Inicialização do backend

No portal, `startServer()` (fim de `server.js`) valida conexão com PostgreSQL, executa `bootstrapDatabase` apenas quando `AUTO_INIT_DB === "true"`, executa `runMigrations`, importa acervo legado por `loadQuizQuestions` e inicia o servidor HTTP. **Mesmo com AUTO_INIT_DB=false, existem tarefas de banco na inicialização**. Nunca usar uma base de produção como ambiente de teste sem backup.

Na Trivia, `src/server.js` valida `TRIVIA_API_KEY`, `TRIVIA_ADMIN_KEY` e credenciais, aguarda o banco, chama `initializeDatabase` (migração e importação condicional) e inicia o Express; `AUTO_IMPORT=if_empty` protege contra reimportação do acervo não vazio.

7. Estado em memória e consequência operacional

As salas do portal são mantidas em `rooms = new Map()`; os temporizadores de desconexão também em memória. `trivia-race.js` mantém `race = new Map()`, `setTimeout` de prazo e transição. Isso significa:

1. Restart/redeploy do portal **interrompe salas e partidas**, mesmo que os usuários continuem registrados no banco.
2. Os dados de conta, sessão e histórico registrados no PostgreSQL sobrevivem ao restart.
3. Rodadas e respostas da Trivia que já chegaram ao schema `trivia` podem sobreviver ao restart da API; mas **o estado da Corrida e a sala** são propriedade do portal e não se reconstituem automaticamente só com o banco.
4. Rodar múltiplas réplicas do portal sem sticky sessions/adapter Socket.IO compartilhado poderá quebrar a consistência. Esse sistema não declara Redis adapter nem persistência distribuída.

8. Dependências externas e risco

- **Twitch/IGDB:** client credential flow e consulta de imagens para personagens/jogos.
- **TMDB:** imagens de atores/personagens de filmes.
- **AniList:** GraphQL para anime/mangá.
- **TVMaze:** APIs de elenco/personagens.

- **Wikipedia/Wikidata/Wikimedia Commons:** descoberta/atribuição de imagens.
- **Google Fonts:** carrega `Chakra Petch` e `Nunito` pela rede no `public/index.html`.
- **Open Trivia Database:** proveniência original de questões na Ronizada Quest, sujeito a atribuição CC BY-SA 4.0.

Fallbacks e prioridades variam por categoria: inspecionar `whoami-images.js` para reproduzir o comportamento por provedor. Falha do provedor externo **não** deve impedir os outros minijogos de serem exibidos.

9. Runtime/deploy recomendado para reprodução

```
# ambiente de desenvolvimento separado para portal
npm ci
npm test
node --check public/script.js
npm start

# em ambiente isolado, executar API separada
npm ci
npm test
npm start
```

Use banco de teste e `.env` individual para cada serviço. Não automatize troca de ID de aplicação, senha ou esquema SQL sem revisar `discloud.config`. Os arquivos podem conter dados licenciados/terceiros que exigem atribuição.

---

02 — MODELAGEM DE DADOS, SQL, ESTADOS E PERSISTÊNCIA

1. Separação lógica

O mesmo PostgreSQL da implantação histórica mantém **duas áreas isoladas**: `public` para Caroninhas Party e `trivia` para Ronizada Quest. O portal consulta `public` diretamente por `pg`; a Trivia API usa conexão e permissão restritas no schema `trivia`. **Não confundir `public.quiz_questions` (recurso legado) com `trivia.questions` (acervo atual da Corrida).**

2. Modelo principal (`database/schema.sql` + migrações)

Tabela	PK	Principais campos / FKs	Uso
`categories`	`id SERIAL`	`name`, `slug`, `emoji`, `display_order`, `active`	Categorias de palavras do Impostor
`difficulties`	`id SMALLSERIAL`	`name`, `slug`, `level` ∈ [1,3]	Níveis usados por palavras e personagens
`words`	`id BIGSERIAL`	`word`, `hint`, `category_id → categories`, `difficulty_id → difficulties`, `active`, `times_used`	Sorteio Impostor; `(word,category_id)` único
`character_categories`	`id SERIAL`	campos semelhantes às categorias de palavras	Filtros Quem Sou Eu?
`characters`	`id BIGSERIAL`	`name`, `image_url`, `category_id → character_categories`, `difficulty_id → difficulties`, `times_used`	Acervo Quem Sou Eu?; `(name,category_id)` único
`users`	`id BIGSERIAL`	`nickname`, `nickname_key` único, `password_hash`, `birth_date`, `avatar_type`, `avatar_id`, `profile_image BYTEA`, MIME	Conta e foto de perfil
`user_sessions`	`token_hash CHAR(64)`	`user_id → users`, `expires_at`, `last_seen_at`	Sessões persistentes em cookie
`match_history`	`id BIGSERIAL`	`round_key` único, `game_type`, `room_code`, `winner_user_id`, `winner_side`, `result_reason`	Histórico de partidas computadas
`match_players`	`(match_id,user_id)`	`game_type`, `won`, `role`, `eliminated`, `attempts_used`, `character_name`	Estatísticas por usuário
`user_game_preferences`	ver migração 004	`user_id`, `game_type`, `preferences`	Preferências salvas por modalidade
`quiz_questions`, `quiz_match_players`	ver migração 002	pergunta, filtros e associação de jogador	Esquema legado de Quiz e Caos — não é fonte da nova Corrida

As migrações incluem `001_v19_accounts.sql`, `002_quiz_caos.sql`, `003_quiz_pawns.sql`, `004_game_preferences.sql`, `005_whoami_expansion_1008.sql`. A última expande o acervo de personagens. O histórico do projeto menciona **1.224 personagens** na operação, mas o número atual deve ser verificado no PostgreSQL porque conteúdo `seed.sql`/migrações e base viva podem divergir.

ER simplificado

```
erDiagram
  CATEGORIES ||--o{ WORDS : possui
  DIFFICULTIES ||--o{ WORDS : classifica
  CHARACTER_CATEGORIES ||--o{ CHARACTERS : possui
  DIFFICULTIES ||--o{ CHARACTERS : classifica
  USERS ||--o{ USER_SESSIONS : possui
  USERS ||--o{ MATCH_PLAYERS : participa
  MATCH_HISTORY ||--o{ MATCH_PLAYERS : contém
  USERS ||--o{ USER_GAME_PREFERENCES : configura
```

3. Detalhes de modelagem importantes

- `users.nickname_key` normaliza nickname por NFKC + minúsculas pt-BR; é único. `nickname` guarda a grafia mostrada.
- `profile_image` é BYTEA no PostgreSQL, com tipo MIME em coluna separada. Foto não é uma URL arbitrária no cadastro da conta.
- Trigger `set_updated_at()` renova `updated_at` em `words`, `characters`, `users`; migrações criam índices para consultas de categoria/dificuldade e histórico.
- `user_sessions` armazena **hash SHA-256 do token de cookie**, não o token puro.
- Histórico de partida é único por `round_key` e associado a jogadores autenticados; visitantes sem conta não têm linha `users` correspondente.
- `getStats()` do código de referência considera os tipos `impostor` e `whoami`. Não afirmar que o progresso da Corrida já integra estatísticas de conta. A Corrida usa pontos efêmeros no servidor.

4. Schema `trivia` — implementação V1.3

Tabela	Campos estruturais	Invariantes
`trivia.questions`	`id TEXT`, pergunta PT/EN, alternativas PT/EN JSONB, gabarito PT/EN, `review_status`, `source_review_status`, `category_pt`, `difficulty`, `question_type`, `content_hash`, `original_record`	só `review_status='approved'` entra no sorteio; `difficulty∈{easy,medium,hard}`, `question_type∈{multiple,boolean}`
`trivia.rounds`	`id UUID`, `room_id TEXT`, `question_id` FK, `shuffled_options JSONB`, `correct_option_id`, `duration_seconds`, `started_at`, `finished_at`	`UNIQUE(room_id,question_id)`; `duration_seconds ∈ [0,30]`
`trivia.round_answers`	`(round_id UUID,player_id TEXT)` PK, `option_id`, `received_at`	uma resposta por jogador/rodada; opção A-D; timestamp registrado pelo banco

```
erDiagram
  TRIVIA_QUESTIONS ||--o{ TRIVIA_ROUNDS : sorteada_em
  TRIVIA_ROUNDS ||--o{ TRIVIA_ROUND_ANSWERS : recebe
```

Nota sobre o acervo: existem 5.247 perguntas na importação da API, e um teste operacional anterior mostrou 5.046 em `approved`, 106 em `blocked` e 95 em `rejected`. As **5.046 foram aprovadas em lote**

para testes, sem revisão factual individual. Esses totais são um retrato histórico, não uma consulta em tempo real. A cópia exata depende do dataset `data/questions.jsonl` e/ou exportação da tabela (sem dados pessoais).

5. Isolamento e permissões

- A API utiliza papel dedicado com permissões sobre `trivia`, historicamente `caroninhas_trivia_api`.
- O portal utiliza sua própria credencial de banco. **Jamais reutilize credencial privilegiada do portal no container Trivia.**
- Criação da role/schema exigiu preparação administrativa uma vez só, fora do bootstrap normal.
- A migração V1.3 cria `trivia.round_answers` de forma aditiva. `public` e as 5.247 questões permanecem preservados.

6. Estado não persistido

Entidade	Onde reside	Se processo reinicia
Salas/códigos/host/membros/ready/spectadores	`Map rooms` em `server.js`	Perdidos
Votos, palavra atual, personagens atribuídos, fases da rodada	`room.currentRound` em memória	Perdidos
Torre/pontos, finalistas, timers de avanço	`Map race` em `trivia-race.js`	Perdidos
Tentativas de recuperação por data de nascimento	`Map` em `auth.js`	Perdidas
Questões e respostas Trivia finalizadas/em aberto	tabelas `trivia`	Persistem, mas portal não reconstrói sala sozinho
Conta e foto	tabelas `public`	Persistem
Cookie de sessão (cliente) / token_hash (banco)	cookie HttpOnly / banco	Persistem dentro da validade

7. Backups e mudança segura

Backup mínimo antes de migrar/deploy: dump das tabelas relevantes + cópia do ZIP atual + `discloud.config` efetivo + inventário dos nomes de variáveis de ambiente (sem valores). Para restaurar, verificar compatibilidade das migrations com a base existente. Não aplicar `sql/000_PREPARAR_SCHEMA_E_USUARIO.sql` repetidamente nem truncar tabelas por conveniência. O bootstrap do portal pode alterar a base mesmo quando `AUTO_INIT_DB` não está habilitado.

8. SQL de auditoria seguro (leitura)

```sql
SELECT schemaname, tablename FROM pg_tables
WHERE schemaname IN ('public','trivia') ORDER BY 1,2;

SELECT review_status, count(*) FROM trivia.questions GROUP BY review_status;
SELECT category_pt, count(*) FROM trivia.questions
WHERE review_status = 'approved' GROUP BY category_pt ORDER BY 1;

SELECT count(*) AS usuarios FROM public.users;
SELECT count(*) AS personagens FROM public.characters;
```

Use conta que possua permissões para essas leituras. Nunca anexar dumps com `password_hash`, `profile_image`, `token_hash` ou dados sensíveis à documentação enviada a outras IAs.

---

03 — CONTRATOS DE INTEGRAÇÃO: REST + SOCKET.IO

1. Convenções

O frontend chama **REST** com `fetch()` para perfil, configuração e catálogo; utiliza **Socket.IO** para salas e jogos. O servidor usa callbacks de confirmação (`ack`) em ações mutáveis. Identificadores `gameType`: `impostor`, `whoami`, `trivia`. Código de sala: **4 caracteres**, alfabeto servidor `ABCDEFGHJKLMNPQRSTUVWXYZ23456789` (sem caracteres visualmente confusos). URLs de entrada: `/impostor/ABCD`, `/quem-sou-eu/ABCD`, `/corrida-do-conhecimento/ABCD` quando código é conhecido.

O portal envia `room-updated` para membros da sala e `rooms-list` para todos quando muda o estado. Clientes nunca escolhem quem é o impostor, qual resposta está correta, o placar ou quem venceu.

2. Rotas HTTP do portal

Método	Rota	Responsabilidade / formato	Autenticação
GET	`/` e `/impostor[/ABCD]`, `/quem-sou-eu[/ABCD]`, `/corrida-do-conhecimento[/ABCD]`	entrega `public/index.html` como shell único	pública
GET	`/quiz-e-caos[/ABCD]`	302 para `/`, modalidade removida	pública
GET	`/api/game-config`	categorias/dificuldades de palavras do Impostor	pública
GET	`/api/whoami-config`	categorias/dificuldades de personagens	pública
GET	`/api/whoami-image`	descoberta de imagem com parâmetros do personagem/categoria	pública; sujeito às credenciais servidor/provedores
GET	`/api/rooms`	`{rooms:[...]}`	pública
GET	`/api/trivia/categories`	proxy da Ronizada Quest; `{categories:[{id,name,available}],defaults:[...],difficulties:[...],durationSeconds:15}`	pública, **chave só backend**
GET	`/api/auth/me`	sessão atual, perfil e estatísticas	cookie opcional
POST	`/api/auth/register`	criar conta com nickname/senha/data nascimento	pública
POST	`/api/auth/login`	entrar e emitir cookie HttpOnly	pública
POST	`/api/auth/logout`	sair e invalidar sessão	sessão opcional
POST	`/api/auth/recovery/verify`	nickname + data nascimento → token temporário	pública com limite de tentativas
POST	`/api/auth/recovery/reset`	token válido + senha nova	token temporário
PATCH	`/api/profile/nickname`	renomear conta	cookie
PATCH	`/api/profile/password`	alterar senha	cookie + senha atual
PATCH	`/api/profile/avatar`	escolher avatar nativo	cookie

Método	Rota	Responsabilidade / formato	Autenticação
POST	`/api/profile/photo`	foto personalizada (bytes conforme handler)	cookie
GET	`/api/profile/image/:id`	servir foto cadastrada com MIME correto	público, imagem ligada a perfil
GET	`/api/game-preferences/:gameType`	preferências da modalidade	cookie
PUT	`/api/game-preferences/:gameType`	salvar preferências da modalidade	cookie
GET	`/api/profile/stats`	resumo da conta	cookie

Erro comum de réplica: não publicar `/api/v1/rounds` diretamente no portal; essa API pertence ao serviço `ronizada-quest` e deve ser chamada de servidor para servidor. Outra falha comum é copiar a chave `TRIVIA_API_KEY` para `public/script.js`: **proibido**.

Contratos de conta (formas principais)

```
// POST /api/auth/register
{"nickname":"Jogador123","password":"[SENHA]","birthDate":"2000-04-12"}
// POST /api/auth/login
{"nickname":"Jogador123","password":"[SENHA]"}
// GET /api/auth/me (quando logado)
{"authenticated":true,"user":
        {"id":123,"nickname":"Jogador123","avatarKey":"dog","avatarType":"preset","avatarId":"dog","avatarUrl":"/
        avatars/dog.png"},"stats":{"totalGames":0,"totalWins":0}}
```

O exemplo contém **campos representativos**, não uma captura completa obrigatória de resposta; use `auth.js` para o payload exato e tratamentos de erros.

3. Formato público de sala (`publicRoom`)

```
{
  "code":"ABCD", "gameType":"trivia", "hostToken":"[token-local]",
  "isPrivate":false, "spectatorCount":0,
  "hostName":"Roni", "hostAvatar":"dog",
  "players":[{"token":"[token-local]","id":"[socket-
        id]","name":"Roni","avatar":"dog","registered":false,"ready":true,"connected":true,"eliminated":false}],
  "playersCount":1,"connectedCount":1,"readyCount":1,"allReady":false,
  "started":false,"roundPhase":null,"createdAt":0,
  "category":{},"difficulty":{},"allFinish":false,
  "triviaSettings":{"categories":["História"],"difficulty":"random","durationSeconds":15}
}
```

Não confiar cegamente em `hostToken` recebido do navegador: toda ação privilegiada valida token efetivamente anexado a `socket.data.playerToken`, código de sala e papel da conexão.

4. Socket.IO — comandos do cliente para servidor

Evento	Payload de entrada (mínimo/mais comum)	Papel e restrições
`get-rooms`	callback	qualquer conexão

Evento	Payload de entrada (mínimo/mais comum)	Papel e restrições
`room-preview`	`{code,gameType}`	qualquer
`resume-session`	`{playerToken,roomCode,sessionSecret}`	jogador pré-admitido
`abandon-session`	sessão existente	vínculo prévio
`create-room`	`{name,avatar,playerToken,category,difficulty,...}`	visitante/conta
`create-whoami-room`	`{name,avatar,playerToken,category,difficulty,allFinish,...}`	visitante/conta
`create-trivia-room`	`{name,avatar,playerToken,categories,difficulty,durationSeconds,isPrivate,roomPassword}`	visitante/conta
`join-room`	`{name,code,playerToken,avatar,gameType,roomPassword,sessionSecret}`	qualquer
`set-ready`	`{code,ready}`	membro da sala
`start-game`	`{code}`	host Impostor
`start-impostor-vote`	`{code}`	host Impostor
`cast-impostor-vote`	`{code,targetToken}`	jogador votante
`start-whoami-game`	`{code}`	host Quem Sou Eu?
`whoami-review-vote`	`{code,targetToken,vote:'approve'`	'reroll'}`
`whoami-reveal-self`	`{code}`	próprio jogador
`whoami-guess`	`{code,guess}`	próprio jogador

Evento	Payload de entrada (mínimo/mais comum)	Papel e restrições
`new-whoami-round`	`{code}`	host, regras do modo
`new-round`	`{code}`	host Impostor
`start-trivia-game`	`{code}`	host Corrida
`trivia-answer`	`{code,roundId,optionId}`	jogador elegível
`restart-trivia-game`	`{code}`	host após vitória
`spectator-get-state`	`{code}`	espectador
`close-room`	`{code}`	host
`leave-room`	callback	participante

Contrato de erro predominante: `callback({ok:false,error:"..."})`; êxito `callback({ok:true,...})`. Nem todo evento usa o mesmo objeto de retorno: consulte os handlers para o formato exato.

5. Socket.IO — mensagens servidor para cliente

Evento	Quando	Payload / preocupação
`rooms-list`	ingresso e atualização	lista filtrável por `gameType`
`room-updated`	ready, membros, lobby etc.	sala pública
`host-transferred`	transferência de controle	identidade novo anfitrião
`room-closed`	fim administrativo	razão e gameType
`game-started`	Impostor	**payload personalizado por jogador**: a palavra não vaza ao impostor
`impostor-vote-started`, `impostor-vote-progress`, `impostor-tiebreak-started`, `impostor-player-eliminated`, `impostor-vote-finished`	votação/eliminação	status, candidatos, vencedores; nunca antes da fase correta
`whoami-review-started`, `whoami-review-updated`, `whoami-character-rerolled`, `whoami-review-complete`	conferência dos personagens	cuidado para não revelar próprio personagem antecipadamente

Evento	Quando	Payload / preocupação
`whoami-game-started`, `whoami-guess-announced`, `whoami-status-updated`, `whoami-game-finished`, `whoami-back-to-lobby`	partida de personagens	resumos, tentativas, fim
`trivia-state`	toda mudança na corrida	fase, pergunta **sem gabarito antes do resultado**, scores e estado geral
`trivia-my-state`	resposta do participante	`{answered,roundId}` específico, em socket pessoal
`trivia-error`, `trivia-back-to-lobby`	falha/revanche	erro ou retorno
`spectator-state`, `spectator-promoted`	assistência e retorno	estado público e promoção para jogar
`back-to-lobby`	nova rodada Impostor	reabre sala

6. Reconexão e segurança da admissão

O cliente armazena `playerToken`, `roomCode` e `sessionSecret` em **`sessionStorage`**. Na reconexão, chama `resume-session`. O portal mantém prazo de desconexão `RECONNECT_GRACE_MS = 30000` ms antes da remoção definitiva; há lógica de transferência de anfitrião/remoção de desconectado. A identidade de conta vem de cookie de sessão validado no backend e **tem precedência** sobre nome/avatar enviados pelo cliente.

`join-room` verifica senha de sala privada com hash `scrypt`; existe limite de **8 tentativas por IP em 60 segundos** para essa verificação, de acordo com o handler de referência. A separação entre `playerToken` e sessão autenticada é essencial: não usar o token do navegador como prova de propriedade de uma conta.

7. Corrida — estado Socket.IO (`trivia-state`)


```
{
  "code":"ABCD","phase":"question","roundNumber":3,"tiebreak":false,
  "players":[{"token":"P1","name":"Roni","avatar":"dog","connected":true,"score":2,"finalista":false}],
  "question":{"roundId":"[UUID]","question":"Pergunta PT-BR", "options":
        [{"id":"A","text":"Alternativa"}],"category":"História","difficulty":"easy","startedAt":"[ISO]","expiresAt":"[ISO]"
  "answerCount":0,"eligibleCount":2,"result":null,"winnerToken":null
}
```


Fases observáveis: `starting`, `loading`, `question`, `resolving`, `results`, `finished`, `error`. Resultado público só quando a Trivia API tiver finalizado. `trivia-my-state` é usado para restaurar o bloqueio do botão ao reconectar.

8. Erros que outra IA precisa evitar

1. Misturar `Socket.IO` e WebSocket puro sem adaptar protocolo/acks/salas.
2. Trocar `playerToken` por `socket.id`; `socket.id` muda ao reconectar.
3. Mostrar gabarito para todos antes do fim da pergunta.
4. Aceitar posição da torre, `winnerToken` ou pontuação vinda do cliente.
5. Alterar os nomes dos eventos sem migrar cliente e servidor juntos.
6. Não transferir/encerrar sala após desconexões e abandono de host.
7. Reiniciar sem redirecionar clientes e perder estados sem tratamento de erro.

Localização do contrato exato: `server.js` (handlers e eventos), `trivia-race.js` (eventos Corrida), `public/script.js` (emissores e listeners) e `auth.js` (REST).

---

04 — ESPECIFICAÇÃO DE PRODUTO, REGRAS E MÁQUINAS DE ESTADOS

1. Padrão transversal a todas as modalidades

O usuário chega à biblioteca (`menu-screen`), seleciona jogo, visualiza tela de criação/entrada, informa identidade (ou usa conta), cria sala por código de quatro caracteres ou entra em uma sala, espera no lobby, marca *pronto*, host inicia somente após critérios do modo, o backend distribui o estado específico e a partida termina em tela/estado de resultado. Salas podem ser públicas ou protegidas por senha; usuários podem reconectar durante tolerância de **30 segundos**; espectadores são tratados separadamente. A sala ativa impede um mesmo token de ingressar simultaneamente em várias salas.

Padrão entre telas: biblioteca → home de modalidade → lobby → partida → resultado → novo lobby ou biblioteca. A interface usa a mesma janela retrô, identidade/avatares e componentes, diferenciando apenas controles particulares do jogo.

Critérios de entrada e prontidão

Jogo	Mínimo de participantes (`publicRoom` / start)	Para iniciar
Impostor	3	host + todos conectados/prontos; palavra sorteável
Quem Sou Eu?	2	host + todos conectados/prontos; personagens suficientes
Corrida do Conhecimento	2	host + todos conectados/prontos; filtros e API válidos

2. Impostor

Objetivo

Um participante é escolhido aleatoriamente como **impostor**; os demais recebem palavra secreta da rodada. O impostor recebe uma experiência/indicação diferente e deve disfarçar sua condição. O jogo usa categorias, dificuldade e dicas sem espaços; distribuição por Socket.IO é **personalizada por participante**, não broadcast de segredo.

Estrutura dos dados da rodada (`room.currentRound`)

```
id, startedAt, word, hint, impostorToken,
category, difficulty,
phase, speakingOrder, voteRound, vote,
tiebreakCandidates, lastVoteCounts,
votedOutToken, eliminatedTokens,
winnerSide, resultReason, winnerToken
```

Fases e transições

```
flowchart TD
  L[Lobby: todos prontos] --> S[Host start-game\nsorteio palavra/impostor]
  S --> D[discussion\nordem de fala]
  D --> V[Host inicia voting]
  V --> C{Apuração}
  C -->|empate| T[tiebreak\ncandidatos reduzidos]
  T --> V
```

```
  C -->|eliminado| E[eliminação / decisão de fim]
  E -->|continua| D
  E -->|vitória de lado| F[finished\nresultado]
  F --> N[Nova rodada/lobby]
```

Regras verificadas no código

- Host só inicia com pelo menos 3 jogadores e todos prontos/conectados.
- Host só abre votação nas fases `discussion` ou `tiebreak`, sem membros desconectados.
- Voto exige candidato válido; **não votar em si mesmo** e **não votar duas vezes**.
- O backend resolve empates por fase específica, administra eliminados e determina `winnerSide` / `resultReason`.
- Cliente recebe progressos e resultado por eventos especializados.
- Estatísticas históricas de conta contemplam papel de impostor/investigador e vitórias.

Para replicar resolução de vitória com exatidão: portar `finishImpostorRound`, `startImpostorVote`, `resolveImpostorVoteIfReady`, `buildImpostorFinishedPayload` e os testes `impostor-vote.test.js` de `server.js`. Não substituir as condições por uma heurística genérica de “maioria”.

3. Quem Sou Eu?

Objetivo

Cada participante recebe um **personagem distinto** do acervo do servidor, com nome e imagem; os demais podem ver o personagem associado ao jogador, mas o próprio jogador não deve recebê-lo antecipadamente na experiência normal. A modalidade utiliza categorias, dificuldade, imagens e uma **conferência colaborativa antes da rodada**. Existe opção `allFinish` (todos concluírem) além do formato clássico.

Estados

```
flowchart TD
 L[Lobby] --> R[review\npersonagens sorteados]
 R -->|aprovações de outros| A{Todos aprovados?}
 R -->|todos pedem troca do alvo| RR[Reroll daquele personagem]
 RR --> R
 A -->|não| R
 A -->|sim| P[playing\npalpites até 3 erros]
 P -->|acertou| C[correct\ncolocação]
 P -->|desistiu| G[gave_up]
 P -->|3 erros| E[out_of_attempts]
 C --> F{Modo clássico ou allFinish?}
 G --> F
 E --> F
 F -->|clássico: primeiro acerto| W[finished]
 F -->|allFinish: todos concluem| W
 W --> L
```

Revisão dos personagens

- `start-whoami-game` sorteia personagens e constrói: `assignments`, `assignmentIds`, `assignmentImages`, `playerStates`, `reviewVotes`, `approvedTargets`, `speakingOrder`.
- Cada jogador avalia o personagem **dos outros** como `approve` ou `reroll`; autoavaliação bloqueada.
- Um alvo só é aprovado com aprovação de todos os outros participantes elegíveis. Troca ocorre se todos os elegíveis solicitarem reroll; novo personagem exclui IDs já utilizados na sala.
- Após aprovar todos os alvos, emite `whoami-review-complete` e inicia fase de jogo.

Palpites, quase acerto e fim

- Cada palpite é classificado por `classifyWhoAmIGuess` em `invalid`, `correct`, `close`, `wrong`.
- Palpite `close` **não consome tentativa**. Algoritmos de normalização canônica, Levenshtein e subconjuntos ordenados existem no servidor; portar esses utilitários e testes para manter tolerância a variantes de nomes.
- Palpite errado consome **uma de três tentativas**. Na terceira, `out_of_attempts` e participante eliminado.
- Acerto produz `placement`, anuncia o acerto e marca vencedor quando for o primeiro. No `allFinish`, acertos e eliminações prosseguem até todos concluírem; no clássico, primeiro acerto encerra a partida.
- `whoami-reveal-self` significa desistir e revelar o personagem, **sem consumir artificialmente tentativas restantes**.
- A interface apresenta personagens, cards com imagens e fallbacks, prévias flutuantes, avisos de palpites, tentativas, quadro de progresso e tela de vencedor.

Imagens e catálogo

`whoami-images.js` decide provedores e aliases conforme categoria/contexto. O código utiliza IGDB, AniList, TMDB, TVMaze, Wikipedia/Wikidata/Commons. Não tratar a URL que o navegador exibiu como única fonte. Há fallbacks para imagens indisponíveis e controle de texto/HTML. Catálogo de personagens vem de `characters` + seed/migração; o usuário relatou 1.224 personagens em versão anterior, valor a validar na base viva.

4. Corrida do Conhecimento — especificação normativa da V20.20.4

Criação de sala

- **24 categorias** carregadas por `/api/trivia/categories`, cada uma com `id` canônico, `name` visual e `available` (número aprovado).
- Tags são botões toggle: clicar seleciona, clicar novamente desmarca; há `Selecionar todas` e `Limpar`.
- Inicialmente selecionadas (se disponíveis): `Conhecimentos gerais`, `História`, `Geografia`, `Ciência e natureza` (mostra “Ciências”), `Cinema`, `Séries e televisão` (“Séries e TV”), `Anime e mangá`, `Videogames`, `Esportes`, `Música`, `Mitologia`, `Livros e literatura` (“Livros”).
- Dificuldade `random` (padrão), `easy`, `medium`, `hard`.
- `durationSeconds`: inteiro **0..30**, padrão **15**; **0 significa sem cronômetro**.
- Sala pode ser privada, com senha de 4 a 64 caracteres.
- Não é permitido criar sem categorias nem com filtros inválidos; servidor valida nomes reais e remove duplicações de forma defensiva.

Fluxo de partida

```
flowchart TD
  L[Lobby\n2+ conectados e todos prontos] --> S[starting]
  S --> C[loading\nsorteio por categoria e dificuldade]
  C --> Q[question\n15 s padrão ou sem relógio]
  Q -->|todos os elegíveis ativos responderam| R[resolving]
  Q -->|prazo terminou| R
  R --> A[POST /finish da Ronizada Quest]
  A --> B{Vencedor único?}
  B -->|não| X[results\nfeedback por 4 s]
  X --> C
  B -->|sim| F[finished\nresultado e campeão]
  C -->|falha de API| E[error\nsem pontuação artificial]
  R -->|falha de API| E
```

Regras concretas

1. **Uma pergunta simultânea** para todos; backend faz chamada `POST /api/v1/rounds` (chave service), envia enunciado, alternativas A-D (ou menos, se T/F) e prazo de acordo com o servidor Trivia.
2. Cada jogador envia `optionId` via evento `trivia-answer`; o backend determina `playerId` pelo `socket.data.playerToken` e chama `POST /rounds/{id}/answers`. Resposta irrevogável.
3. `round_answers` tem PK `(round_id,player_id)` e valida o prazo com `clock_timestamp()` no PostgreSQL. Não aceite timestamp enviado pelo navegador.
4. Backend encerra quando **todos os conectados e elegíveis** responderam ou quando `expiresAt` é atingido. O `setTimeout` local acrescenta ~350 ms de margem, mas a decisão sobre pontualidade pertence ao banco remoto.
5. `/finish` corrige só respostas realmente registradas antes do prazo. Ausentes **não avançam**.
6. Resposta correta: pontuação `min(10,score+1)`; errada ou não enviada: `+0`.
7. Sem vencedor: fase `results` por **4.000 ms**, em seguida rodada automática.
8. **Vencedor exclusivo** na casa 10: `finished`, tela de campeão com avatar, ranking final, número de rodadas e indicação de desempate, quando houver.
9. **Dois ou mais alcançam 10 na mesma rodada:** somente esses tornam-se finalistas e respondem a perguntas extras. Se só um acerta, vence; se vários acertam, apenas esses continuam; se nenhum acerta, repetem os mesmos finalistas. Torres permanecem 10/10.
10. O vencedor e a classificação vêm do estado servidor, não de cálculo navegador. A tela final é mostrada após pequeno intervalo (~1,7 segundo no cliente na versão V20.20.3).
11. Host pode reiniciar para lobby após vitória. Ao reiniciar: `cleanup`, reseta `ready`, promove espectadores quando pertinente, emite `trivia-back-to-lobby`.

Escolha da pergunta

`trivia-race.js` embaralha categorias selecionadas, ordena para favorecer as menos utilizadas na partida (`categoryUses`), e tenta dificuldades em ordem embaralhada quando `random`. Para categoria/dificuldade sem pergunta, tenta outras combinações **apenas quando a API retorna conflito HTTP 409**. `roomId` da API inclui UUID gerado ao iniciar a corrida, o que previne repetição de `question_id` naquela mesma partida por constraint `UNIQUE(room_id,question_id)`.

Acerto e pontuação entre usuários

O resultado remoto é mapeado por `playerId == playerToken`; `correctOptionId` só é revelado quando finaliza. O contador de resposta e a pontuação são atualizados por evento `trivia-state`. `trivia-my-state` permite que a UI continue bloqueando uma segunda resposta após reconexão.

Limitações observadas — não esconder numa réplica

- Progresso Corrida está em memória no portal; ao reiniciar o servidor o estado da partida some.
- A tabela `match_history` / `getStats` do código analisado contabiliza modalidades clássicas; a Corrida **não tem estatística persistente de vitórias de conta equivalente** comprovada.
- Sem cronômetro depende de todos os participantes ativos responderem; desconexão/abandono exige tratamento para não travar a rodada.
- O acervo aprovado em lote pode conter erros de tradução ou gabarito; não tratar aprovação administrativa como revisão científica.

5. Regras de reaproveitamento obrigatório

Toda modalidade **precisa** usar: o mesmo `retro-window`, identidade (conta/visitante), avatar, modal de escolha, validação do host, privacidade, cópia de código, pronto/aguardando, reconexão, modo espectador e notificações. Somente telas e lógica internas devem diferir. Evitar recriar CSS local que sobrescreva tokens do site inteiro.

05 — FRONTEND, NAVEGAÇÃO, DOM, MODAIS, ESTADOS E RESPONSIVIDADE

1. Modelo de frontend: SPA sem framework

`public/index.html` contém **todas as telas pré-renderizadas como seções**; apenas uma tela aparece por vez. `public/script.js` é carregado globalmente e gerencia eventos e fluxo através de funções, objetos de estado e seletores DOM; a função `showScreen(screen)` oculta todos os elementos de `allScreens`, remove classes antigas `screen-mode-*` de `<body>` e adiciona a classe da tela atual (ex.: `screen-mode-trivia-game-screen`). CSS usa essas classes para centralização, limites de viewport e variações de cada modalidade. O frontend não é React nem usa roteador de biblioteca; links `/impostor/ABCD`, etc. são processados por regex e `pendingDeepLink` em `script.js`, enquanto o Express entrega `index.html`.

Existe `MutationObserver` que normaliza o início de textos visíveis com letra maiúscula (excluindo, por exemplo, `.window-title`). Uma reconstrução que altere esta lógica pode mudar a capitalização de títulos e microcopy.

2. Inventário das telas principais

ID HTML	Modalidade	Missão
`menu-screen`	comum	Biblioteca de minijogos, cards, informações de conta/estado
`impostor-home-screen`	Impostor	Criar/entrar, selecionar categoria e dificuldade, privacidade
`lobby-screen`	Impostor	Código, lista, ready, host
`game-screen`	Impostor	Palavra/papel/ordem de fala
`impostor-vote-screen`	Impostor	votação e candidatos
`impostor-result-screen`	Impostor	eliminação/vencedor por lado
`whoami-home-screen`	Quem Sou Eu?	Criar/entrar, filtros, regras, privacidade
`whoami-lobby-screen`	Quem Sou Eu?	Lobby e confirmação
`whoami-review-screen`	Quem Sou Eu?	Revisão colaborativa e reroll
`whoami-game-screen`	Quem Sou Eu?	Cards/nomes/imagens de outros; palpites, status e tentativas
`whoami-winner-screen`	Quem Sou Eu?	Resultado e classificação, considerando allFinish
`trivia-home-screen`	Corrida	Tags, dificuldade, tempo, privacidade, salas abertas e entrada
`trivia-lobby-screen`	Corrida	Código, players, prontidão, início pelo host
`trivia-game-screen`	Corrida	Pergunta + respostas na esquerda; torres na direita; timer
`trivia-winner-screen`	Corrida	Campeão, foto/avatar, ranking, estatísticas e revanche
`spectator-screen`	comum	Status de espectador da sala e progresso observável

Não confundir tela com URL: telas internas mudam via classe `.hidden`, mesmo sem alterar `location.pathname`.

3. Estado de identidade do cliente

`script.js` define `AVATARS` (10 PNGs), `STORAGE_KEYS`, `playerToken` e funções de conta/perfil. Há três possibilidades: (a) visitante com nome e avatar local, (b) conta autenticada com nickname/avatar preset, (c) conta com foto enviada ao banco e servida pelo endpoint de perfil. `syncGameIdentityUI` e funções de preview/modal tornam a identificação **consistente entre os três jogos**. O navegador não pode fazer o backend aceitar nickname falso quando a sessão está autenticada; `resolvePlayerIdentity()` substitui o pedido pela conta verdadeira.

`sessionStorage` e `localStorage`

Chave	Storage	Uso
`caroninhas_player_token`	session	identidade de participação entre reconnects na aba
`caroninhas_room_session_secret`	session	segredo para retomar entrada
`caroninhas_room_code`	session	código atual
`caroninhas_player_name`	local/session conforme função	apelido preferido do visitante
`caroninhas_player_avatar`	local/session conforme função	avatar preferido
chave de preferências da Corrida em `caroninhas_party_settings_v1:trivia`	local	tags, dificuldade, segundos e privacidade quando aplicável

Consultar `public/script.js` para a distribuição exata entre storages: armazenamento não é autenticação de conta.

4. Componentes reutilizáveis de interface

Componente / classe	Estrutura esperada	Semântica
`.retro-window`	caixa com `.window-bar` + conteúdo	moldura desktop, raio e sombra dura
`.window-bar`	`.window-dots` + `.window-title` + `.window-actions`	barra rosa, pontos coloridos e ícones decorativos
`.retro-button` + `.primary` / `.secondary`	`<button>`	CTA, variação de cores, sombra e borda
`.mini-button`	`<button>`	ações secundárias (copiar, limpar, atualizar)
`.retro-input-wrap`	container contendo `<input>` e ícone opcional	caixas de texto, identidade, código e senha
`.retro-select`	`<select>`	dificuldade/categoria; seta integrada sem faixa rosa extra
`.retro-check-option`	checkbox e texto	seleção de privacidade e preferências
`.guest-identity-card`, `.guest-avatar-preview`, `.guest-name-row`	nome/avatar do jogador	**mesmo padrão em todas as modalidades**

Componente / classe	Estrutura esperada	Semântica
`.nav-back`	botão	retorno sem navegação quebrada
`.tiny-label`	microtítulo	kicker retrô, geralmente maiúsculo e espaçado
`.players-panel`, `.ready-summary`, `.waiting-box`	blocos do lobby	participantes, confirmação, aguardo
`.error`, `.toast-message`	texto de status	erro e confirmação acessível
`.screen.hidden`	seção invisível	transição de tela
`.open-rooms-section`	lista + cabeçalho e refresh	descoberta de salas

Modal de avatar

Não recriar um seletor próprio para cada jogo. O mesmo modal lista avatares preset e exibe prévia; avatar personalizado para usuário autenticado é parte do perfil. Dependendo da origem do valor, `getAvatarDefinition()` converte chave `profile:<id>:<versão>` em `/api/profile/image/<id>?v=...` ou busca `/avatars/<animal>.png`.

5. Página de criação da Corrida

Estrutura real de `index.html`:

```
#trivia-home-screen
  .game-page-shell
    .nav-back
    .retro-window.trivia-window
      .window-bar
      .trivia-home-main
        .trivia-intro  (JOGO 03 / título / subtítulo)
        .guest-identity-card (nome e avatar)
        .trivia-config  (grid de duas áreas)
          .trivia-main-settings
            .trivia-categories-card  (título, texto, 24 tags, selecionar/limpar)
            .trivia-join-section  (código, senha, botão entrar)
          .trivia-side-settings
            .trivia-create-settings  (dificuldade, tempo -/+; sala privada; CTA)
            .trivia-open-rooms    (lista de salas e atualizar)
```

O formulário **não** deve criar uma janela escura dentro do portal: os elementos herdam tokens globais. O painel de tags é grupo de botões com `aria-pressed`; nome e avatar aparecem no mesmo componente usado no Impostor/Quem Sou Eu?. O tempo tem botões `trivia-time-minus` e `trivia-time-plus` e um número sempre legível, de 0 a 30.

Tags e preferências

`loadTriviaConfig()` busca `/api/trivia/categories`, converte `defaults` para seleção inicial quando o navegador não tem preferências, mantém lista canônica de ids, salva em localStorage e desabilita criar sala com zero tags ou API indisponível. `displayCategory` substitui somente os rótulos amigáveis; o servidor usa IDs canônicos no filtro SQL.

6. Lobby e corrida em andamento

- Lobby: `.lobby-content`, `.room-code-panel`, `.players-panel`, pronto, iniciar (somente host), configurações selecionadas, lista de jogadores com fotos. O botão do host depende de `allReady`.

- Partida: `.trivia-play-header` com título `Rodada N` e `.trivia-timer`; `.trivia-play-grid` com dois painéis: `.trivia-question-panel` e `.trivia-towers-panel`.
- Painel pergunta: categoria, dificuldade, enunciado, alternativas clicáveis, revelação do gabarito e feedback de acerto/erro; desabilitar após envio.
- Torres: 10 segmentos por jogador empilhados de cima para baixo, mas **preenchimento visual de baixo para cima** (`for level = 10 ... 1`). Avatar e nome em cima; score `N/10`; classes `.advanced`, `.offline`, `.finalista`; cor individual via `--tower-color`.
- Celular: jogo muda para uma coluna, com **torres acima da pergunta**, blocos mais baixos, possível rolagem horizontal **somente no painel das torres**, e alternativas podem virar coluna única em larguras pequenas.

7. Tela de campeão

`#trivia-winner-screen` é uma janela própria, semelhante à apresentação de vencedor de Quem Sou Eu?. Inclui: `.trivia-winner-heading`, `.trivia-winner-hero` com avatar grande e coroa, `.trivia-winner-details`, `.trivia-winner-stats`, `.trivia-ranking-panel` com classificação final, `.trivia-winner-actions` para revanche (host) e volta à biblioteca. A transição sai de `trivia-game-screen` quando o estado `finished` é recebido, após cerca de 1,7 s, preservando feedback do último acerto.

8. Telas de Impostor / Quem Sou Eu? — o que replicar

- Preservar cabeçalho e moldura, campos de categoria/dificuldade, nome/avatar, sala pública/privada, código/entrar e listas.
- Impostor: cartões secretos e avisos de impostor, votação em telas dedicadas e placar/fim.
- Quem Sou Eu?: cards de personagens, imagética e fallbacks, revisão coletiva, modais de imagem e preview flutuante; avisos em fila, palpites e classificação de todos.
- O site tem área de conta: login/cadastro, recuperação, perfil, troca de nickname/senha/avatar/foto e estatísticas. Uma réplica só das três telas de jogo **não é réplica completa**.

9. Acessibilidade e experiência real

Há `aria-label`, `aria-live`, `aria-pressed` e botões nativos em elementos específicos; replicar ao menos os existentes. Melhorias desejáveis não confirmadas no código: auditoria WCAG 2.2, testes completos por teclado/leitor de tela, políticas CSP e redução de movimento ampla. Marcar como trabalho futuro, não como recurso já presente.

10. Fluxo de telas ao implementar nova modalidade

Reutilize o padrão completo: inclusão em `renderGames` (card da biblioteca), DOM específico em `index.html`, registro em `allScreens`, entradas de `showScreen`, deep link, eventos de lobby, identidade sincronizada, lista de salas, privacidade/ready/host, reconexão e saída, telas de resultado. Não adicionar JS que force um tema/cor local de forma global. Não remover as três modalidades existentes.

---

06 — DESIGN SYSTEM VISUAL: IDENTIDADE, TIPOGRAFIA, CORES, MEDIDAS E GRID

1. Linguagem de interface

O Caroninhas Party usa **retro desktop lúdico**, influência de interfaces de janelas antigas/pixel art, mas com cartões e campos contemporâneos, legibilidade e cores pastel. O fundo tem grid rosa-lilás claro; conteúdo aparece em janelas creme com barras de título rosa, contorno grafite de 3 px, cantos arredondados e sombras **duras deslocadas** que simulam elementos de brinquedo/adesivo. Estrelas e coração decorativos aparecem no plano de fundo; detalhes em amarelo, lima, menta e laranja contrabalançam o rosa.

Não usar: dark mode por jogo, glassmorphism genérico, gradientes neon, cartões sem contorno, botões gigantes inconsistentes, fontes pixel ilegíveis para números pequenos, dashboards SaaS escuros, pictogramas de videogame futurista. A beta escura inicial da Corrida foi explicitamente rejeitada pelo usuário.

2. Tokens CSS globais (valores reais de `public/style.css`)

```css
:root {
  --bg: #ead7e2;
  --grid: rgba(116, 84, 92, 0.10);
  --cream: #f7f1df;
  --cream-2: #fff9e9;
  --cream-3: #eadfbe;
  --ink: #2e2524;
  --muted: #786a65;
  --line: #2f2927;
  --pink: #f3a6b9;
  --pink-deep: #df7f9a;
  --orange: #ef9c63;
  --lime: #d8e87e;
  --yellow: #f2d36f;
  --mint: #91c9b6;
  --shadow: #2e2524;
  --soft-shadow: rgba(46, 37, 36, 0.15);
}
```

Função	Token	Valor	Observação visual
Plano de fundo	`--bg`	`#ead7e2`	rosa/lilás desaturado
Grade de fundo	`--grid`	`rgba(116,84,92,.10)`	linhas horizontais/verticais
Janela	`--cream`	`#f7f1df`	creme amarelado
Formulários	`--cream-2`	`#fff9e9`	creme quase branco
Faixas secundárias	`--cream-3`	`#eadfbe`	bege profundo
Texto	`--ink`	`#2e2524`	quase preto amarronzado
Texto auxiliar	`--muted`	`#786a65`	taupe
Contorno	`--line`	`#2f2927`	mesmo caráter do texto
Barra / CTA	`--pink`	`#f3a6b9`	rosa-claro
Ênfase rosa	`--pink-deep`	`#df7f9a`	destaque de título
Ação/alerta	`--orange`	`#ef9c63`	laranja-pêssego

Função	Token	Valor	Observação visual
Badges e destaques	`--lime`	`#d8e87e`	verde-amarelado
Destaque de botão/role	`--yellow`	`#f2d36f`	amarelo claro
Estado positivo	`--mint`	`#91c9b6`	menta
Sombra	`--shadow`	`#2e2524`	deslocada, borda firme

Fundo quadriculado

```
body {
  margin: 0;
  min-height: 100vh;
  background-color: var(--bg);
  background-image:
    linear-gradient(var(--grid) 1.5px, transparent 1.5px),
    linear-gradient(90deg, var(--grid) 1.5px, transparent 1.5px);
  background-size: 42px 42px;
  color: var(--ink);
  font-family: "Nunito", sans-serif;
}
```

Exibir decorações como conteúdo não interativo em `position: fixed`, `pointer-events: none`, camada atrás da UI (estrelas/coração e outras formas), sem esconder campos nem competir com a leitura.

3. Tipografia exata da V20.20.4

O site utiliza **Chakra Petch** (títulos e rotulagem retrô) e **Nunito** (corpo, explicações e vários formulários). A fonte **Pixelify Sans era usada nas versões anteriores, mas foi substituída globalmente por Chakra Petch na V20.20.4** por problemas de legibilidade, principalmente dos numerais como “15”. Não usar Pixelify como família principal ao replicar V20.20.4.

Papel	Fonte	Peso típico	Observação
Título de jogo	Chakra Petch	700/800	grande, alto contraste, não espremido
Barra de janela, kicker, badge	Chakra Petch	700	letter-spacing em microtítulos
Botões retrô	Chakra Petch	700/800	preserva estilo sem número ilegível
Numerais (timer, pontuação)	Chakra Petch	700	legível no 15, 0, 10
Textos longos e descrições	Nunito	700/800 ou regular conforme seletor	corpo amigável arredondado
Alternativas/inputs	Nunito/Chakra Petch conforme classe	~700/800	preservar regra da classe original

As fontes são carregadas pelo Google Fonts em `index.html` via `<link>`, não pela distribuição de fontes locais. **Não incluir arquivos de fonte** na documentação/distribuição. Se a importação externa falhar, os fallbacks são `sans-serif`.

4. Estrutura dimensional da janela

Valores reais de referência:

```
.retro-window, .desktop-widget {
  overflow: hidden;
  border: 3px solid var(--line);
  border-radius: 16px;
  background: var(--cream);
  box-shadow: 8px 9px 0 var(--shadow);
}
.window-bar {
  min-height: 42px;
  display: grid;
  grid-template-columns: 1fr auto 1fr;
  align-items: center;
  padding: 0 14px;
  border-bottom: 3px solid var(--line);
  background: var(--pink);
}
.window-dots { display: flex; gap: 7px; align-items: center; }
.dot { width: 11px; height: 11px; border: 2px solid var(--line); border-radius: 50%; }
```

Barra de título: três pontos de cor rosa, laranja e lima à esquerda, nome da janela estilo `*.exe` ao centro, três ícones decorativos à direita. Sempre manter a proporção, linha de separação forte e fundo rosa. `.window-bar.compact` usa min-height de 34 px.

`.app` base: largura `min(1180px, 92vw)`, centrada; padding vertical de 58/72 px na página inicial. Em modo de jogo e viewport desktop, regras específicas centralizam `.app` e limitam altura próxima a `100dvh`. Janelas individuais podem ter `game-page-shell` entre aproximadamente 620 px (modos antigos) e 1120 px/área de Corrida, dependendo da classe de tela.

5. Componentes — geometria, hierarquia e interação

Elemento	Forma	Tratamento
Janela principal	retângulo arredondado (`~16px`)	fundo creme, borda 3px, sombra sólida deslocada 8×9px
Janela modal/ card	mesmo contorno gráfico	fundo creme/papel, recortes e linhas divisórias
Botão primário	retângulo com cantos arredondados	rosa/lima conforme contexto, borda escura, sombra dura de poucos px
Botão secundário	semelhante com fundo creme	menos destaque; nunca menor verticalmente que input adjacente
Input	fundo creme-claro, contorno escuro e raio 12-16px	estado focus, placeholder muted, valor legível
Select	seta integrada	**sem barra rosa decorativa vertical** na V20.20.4
Checkbox	bloco normalizado	**sem faixa rosa ao lado** na V20.20.4
Tag categoria	pílula, contorno escuro	selecionada em rosa com check; desmarcada creme com símbolo `+`
Cartão de player	avatar, nome, status, badge	sem identidade inventada
Torre	10 retângulos empilhados	preenchimento baixo→cima, cor do jogador, texto N/10
Ícones	sprite SVG em `index.html`	traço consistente; símbolos infantis/decorativos pontuais

Refinamentos formalizados na V20.20.4: inputs de código e senha e CTA Entrar na Corrida têm **52 px de altura externa**, alinhados verticalmente; o número do tempo usa Chakra Petch; linha rosa decorativa de `.retro-select` / `.retro-check-option` foi removida. Esses detalhes foram explicitamente solicitados pelo usuário e são **invariantes de aceitação**.

6. Sistema de espaçamento e proporção

O CSS não declara um sistema único de tokens `--space-*`; contém valores concretos contextuais. Para réplica pixel aproximada, medir a captura visual de referência e reutilizar os números do CSS em vez de inventar um design system de 8px sem conferir. Padrões recorrentes: gaps de 7–20px, paddings de 12–30px, contornos 2–3px, raios 10–18px, microtítulos 10–13px, texto geral 13–17px, títulos de jogos `clamp()` entre ~28 e ~56px. Valores exatos variam pela tela.

7. Especificidade, cascata e regressão

`public/style.css` possui mais de 6.000 linhas e dezenas de regras sobrepostas de versões históricas. O visual real é determinado pela **cascata final** e classes do `body`, não pela primeira declaração do seletor. Para uma réplica consistente:

1. Capturar estilos computados em cada tela (Chrome DevTools); comparar com seletor e `@media` ativo.
2. Não pegar CSS da V20.20 beta escura ou da V20.20.1 e tratá-lo como final.
3. Não eliminar estilos do Impostor ou Quem Sou Eu? ao simplificar classes de Corrida.
4. Alterar apenas regras específicas ou consolidar a cascata com testes de regressão visual antes de publicar.

8. Responsive e viewport

Breakpoints visíveis no arquivo incluem **390**, **720/721**, **980** px e outros associados às telas antigas. A lógica geral é:

- **Desktop**: biblioteca em grade; janelas centralizadas; home da Corrida com categorias à esquerda e configurações/rooms à direita; tela da partida com pergunta ~60% e torres ~40%.
- **Tablet estreito**: configurações colapsam progressivamente; controles e cartões reorganizados.
- **Mobile**: uma coluna; janelas com largura responsiva; torres antes da pergunta e lista interna com overflow horizontal quando necessário; inputs e botões viram linhas verticais; rolagem habilitada conforme conteúdo.
- **Rolagem:** preferência é caber na viewport desktop, mas telas pequenas/zoom acessível podem ter scroll **interno seguro**. Não bloquear scroll global deixando o botão de criar sala inacessível.

9. Identidade por modalidade

Impostor usa detalhes amarelos na `.window-bar.impostor-bar`; Quem Sou Eu? enfatiza cartões com imagem; Corrida usa mesmas superfícies/rosa e novas tags/torres. Diferenças de modalidade não devem criar um **novo sistema de cores**.

10. Referências de UI e capturas

As imagens em `referencias-visuais/` são referências de composição/escala, com duas distinções: `captura-*` deriva de imagem enviada pelo usuário, enquanto `preview-*` é prévia de desenvolvimento. Em qualquer conflito visual, priorize a **versão realmente publicada que você estiver reconstruindo** e os estilos efetivos da V20.20.4. Nenhuma imagem de referência substitui HTML funcional nem contagem do banco.

Checklist visual de fidelidade

- [ ] Rosa quadriculado, creme, borda 3px, sombra dura e janela rosa.
- [ ] Chakra Petch nos títulos/controles de destaque e Nunito no corpo.
- [ ] Mesmo componente nome/avatar em todos os jogos, incluindo foto de conta.
- [ ] Tags pílula clicáveis e 24 categorias quando API disponível.
- [ ] Número 15 sempre legível com controle –/+, sem setas sobrepostas.
- [ ] Input código, input senha e botão Entrar visualmente alinhados.
- [ ] Sem faixas rosas residuais em select/checkbox.
- [ ] Jogo usa 10 blocos por torre, dados atualizados sem manipular score no browser.
- [ ] Tela final com coroa, avatar, classificação e ação de revanche.
- [ ] Mobile sem corte/estouro horizontal de página.

---

07 — RONIZADA QUEST V1.3 — SERVIÇO DE PERGUNTAS, PRAZOS E SORTEIO

1. Função e limites

Ronizada Quest / Caroninhas Trivia API é um **segundo aplicativo Express independente**. Ela mantém questões, aprovadores, categorias, rodadas, gabaritos privados e respostas registradas no tempo. **Não** é o frontend da Corrida; **não** gerencia lobbies, avatares, host, prontos nem torres. O portal Caroninhas Party é o orquestrador multiplayer que consome a API com `x-api-key` a partir do backend.

A API expõe `/health` para verificação e `/api/v1/*` para operações de serviço. A versão estudada é **1.3.0**.

2. Contratos REST exatos de alto nível

Método	Rota	Header obrigatório	Retorno/semântica
GET	`/`	nenhum	service, online, version
GET	`/health`	nenhum	200 quando banco OK, 503 quando indisponível
GET	`/api/v1/stats`	`x-api-key`	`{total,by_status}`
GET	`/api/v1/categories`	`x-api-key`	`{categories:[{name,total,available}]}`
POST	`/api/v1/rounds`	`x-api-key`	201 + pergunta/opções sem resposta correta
GET	`/api/v1/rounds/:id`	`x-api-key`	pergunta recuperável, `expiresAt`, `expired`, `finished`, sem gabarito
POST	`/api/v1/rounds/:id/answers`	`x-api-key`	201 resposta pontual; 409 duplicada/encerrada; 410 atrasada
POST	`/api/v1/rounds/:id/finish`	`x-api-key`	200 gabarito + resultado das respostas previamente registradas
GET	`/api/v1/admin/questions`	`x-admin-key`	consulta de questões para revisão
PATCH	`/api/v1/admin/questions/:id/status`	`x-admin-key`	alteração do status de revisão

A chave de serviço (`TRIVIA_API_KEY`) e a administrativa (`TRIVIA_ADMIN_KEY`) devem ser **distintas**. `src/app.js` valida com `secureMatch` e middleware do Express; resposta típica a chave ausente/inválida: `401`.

3. Criar rodada

```
POST /api/v1/rounds
x-api-key: <SECRET_DO_SERVIDOR>
Content-Type: application/json

{"roomId":"caroninhas-uuid-exemplo","category":"Videogames","difficulty":"medium","durationSeconds":15}
```

Resposta representativa:

```
{
 "roundId":"00000000-0000-4000-8000-000000000000",
 "questionId":"[id-opentdb]",
 "question":"Em que jogo... ?",
 "category":"Videogames","difficulty":"medium","type":"multiple",
 "options":[{"id":"A","text":"..."},{"id":"B","text":"..."},{"id":"C","text":"..."},{"id":"D","text":"..."}],
 "durationSeconds":15,"startedAt":"2026-10-24T12:00:00Z","expiresAt":"2026-10-24T12:00:15Z",
 "attribution":"OpenTDB"
}
```


O exemplo é esquemático (não uma pergunta real). `roomId` precisa corresponder a `[a-zA-Z0-9_:.-]{1,120}`. Categoria é o **nome canônico do banco**, não um apelido de interface. `difficulty` aceita `easy`, `medium`, `hard`; ausência usa o filtro opcional da API quando permitido. `durationSeconds` deve ser inteiro de 0 a 30.

Garantias SQL na criação

A API inicia transação, utiliza `pg_advisory_xact_lock(hashtext(roomId))`, recusa sala com rodada ainda aberta e seleciona pergunta `review_status='approved'` que ainda **não** apareceu no mesmo `room_id`. `ORDER BY RANDOM() LIMIT 1`. Monta alternativas com ID A-D e as embaralha; grava `correct_option_id` **somente no banco**; `UNIQUE(room_id,question_id)` evita repetição daquela questão dentro da partida. Falta de pergunta aprovada para filtro retorna `409`.

4. Enviar resposta

```
POST /api/v1/rounds/<roundId>/answers
x-api-key: <SECRET_DO_SERVIDOR>
Content-Type: application/json

{"playerId":"[TOKEN_RESOLVIDO_NO_SERVIDOR]","optionId":"C"}
```

Atenção: `playerId` é fornecido pelo **backend do portal** após validar socket, sala e fase; o navegador envia só `optionId` ao portal. A API inicia transação, bloqueia a linha da rodada (`FOR UPDATE`), verifica alternativa existente e executa `INSERT ... SELECT` com condição temporal:

```
AND (
  r.duration_seconds = 0
  OR clock_timestamp() < r.started_at + (r.duration_seconds * INTERVAL '1 second')
)
```

Há `ON CONFLICT (round_id,player_id) DO NOTHING` e UNIQUE/PK para uma resposta por jogador por rodada. A resposta é **imutável** após registrada. Código HTTP:

- `201`: registro aceito, resposta contém `registered:true` e `receivedAt` do banco.
- `409`: já respondeu ou rodada fechada.
- `410`: após prazo, nenhuma resposta registrada.
- `400`: ID inválido, alternativa inválida, corpo inválido.
- `401`: autenticação de serviço ausente/inválida.
- `404`: rodada inexistente.

Com `durationSeconds=0` a condição temporal aceita até o `/finish` encerrar a rodada manualmente.

5. Encerrar e pontuar

```
POST /api/v1/rounds/<roundId>/finish
x-api-key: <SECRET_DO_SERVIDOR>
```

```
Content-Type: application/json

{}
```

Não colocar `{answers:[...]}` nessa rota. A V1.3 **removeu** a possibilidade de entregar respostas junto ao fechamento; faz correção apenas do que já está em `trivia.round_answers`. Resposta contém `correctOptionId` e `results:[{playerId,optionId,correct}]`. Quem não respondeu não aparece nessa lista; o portal o trata como erro/ausência (`+0`). Isso garante que um encerramento tardio não valide respostas enviadas depois do relógio.

6. Comunicação Portal → Trivia

`trivia-race.js` implementa `api(endpoint, options)`: base `TRIVIA_API_URL` (fallback histórico `https://ronizada-quest.discloud.dev`), timeout 12 segundos via `AbortSignal.timeout(12000)`, `x-api-key` e `content-type: application/json` no `fetch`. `/api/trivia/categories` do portal consulta a API e faz cache **em memória por 60 segundos**. Falha de conexão apresenta HTTP 503 com texto informativo (não expõe segredo).

Na Corrida, `nextRound` escolhe categorias/dificuldades; `finish` mapeia resultados por token, incrementa no máximo até 10, decide desempate e emite atualização Socket.IO. Falhas na API não devem virar acerto falso; a fase passa a `error`.

7. Categorias e regras da configuração

A API disponibiliza até **24 categorias** conforme as questões PT-BR e status de revisão. Um filtro por nome precisa ser exatamente `category_pt` do banco. O portal define três aliases de exibição: `Ciência e natureza → Ciências`, `Séries e televisão → Séries e TV`, `Livros e literatura → Livros`. O backend nunca envia o alias no lugar do identificador canônico. É possível marcar todas, retirar todas, ou selecionar qualquer combinação não vazia.

8. Revisão e origem do acervo

Questões oriundas do Open Trivia Database, com derivados traduzidos PT-BR. A origem é citada como **CC BY-SA 4.0** nos documentos da API; manter atribuição e cumprir termos aplicáveis. Há dados em `data/questions.jsonl`, pipeline de importação em `src/importer.js` e scripts `db:import`, `db:check`. O conteúdo contém estados `pending`, `approved`, `blocked`, `rejected`; somente `approved` participa de rodadas. O lote de 5.046 questões aprovado para ensaio **não** passou por revisão factual individual. Se outra IA reproduzir o site, não transformar esse status operacional em certificação de qualidade.

9. Deploy e preservação de dados

- App Discloud isolado: `ronizada-quest`.
- DB PostgreSQL existente, schema `trivia`, role limitada `caroninhas_trivia_api`, rede privada com VLAN no cenário documentado.
- `.env` individual: `TRIVIA_API_KEY`, `TRIVIA_ADMIN_KEY`, credenciais do banco e `AUTO_IMPORT=if_empty` quando cabível.
- Migração aditiva V1.3 cria `trivia.round_answers`. Não apagar nem reimportar as perguntas ao atualizar somente a lógica.
- Verificar `/health`, `/api/v1/stats` (via backend autorizado) e um ciclo de rodada/duplicata/410/finish.

10. Casos de teste essenciais

1. Criar rodada de 15s, responder de dois jogadores dentro do prazo; repetir resposta do primeiro deve dar `409`.
2. Criar nova rodada de 15s, tentar responder depois do prazo; deve dar `410`; `/finish` retorna `results:[]` se ninguém respondeu.
3. Criar sem cronômetro (`0`); resposta aceita depois de 15s **desde que rodada não tenha sido encerrada**.
4. `GET /rounds/:id` nunca revela `correctOptionId`.
5. Duas rodadas com mesmo `roomId` não repetem a mesma questão; não criar duas rodadas abertas simultaneamente para mesmo roomId.
6. Filtro por categoria/dificuldade esgotado retorna `409`, portal tenta outras combinações válidas.
7. Verificar erro de API/restart sem pontuação local incorreta.

---

08 — SEGURANÇA, CONTAS, AMBIENTE, OPERAÇÃO E INCIDENTES

1. Modelo de autenticação existente

O sistema de contas reside em `auth.js` e banco `public`. Nickname permitido: de **3 a 20 caracteres**, letras ASCII, números, ponto e sublinhado; reserva `admin`, `moderador`, `sistema`, `caroninhas`. A comparação de unicidade usa normalização **NFKC + `toLocaleLowerCase('pt-BR')`**. Senha permitida: de **6 a 72 caracteres**. Data de nascimento no formato `YYYY-MM-DD` e coerência temporal básica.

As senhas são armazenadas usando **scrypt** com salt aleatório de 16 bytes, chave derivada de 64 bytes. `password_hash` segue `scrypt$<salt_base64>$<hash_base64>`. Verificação usa `crypto.timingSafeEqual` e scrypt assíncrono.

Sessão

- Login gera token aleatório de 32 bytes, exposto apenas como cookie `caroninhas_session`.
- Banco guarda `SHA256(token)` em `user_sessions.token_hash`, nunca o token bruto.
- Cookie: `HttpOnly`, `SameSite=Lax`, `Path=/`, `Max-Age` 30 dias, `Secure` quando HTTPS/proxy sinalizam conexão segura.
- `getUserFromRequest` e `getUserFromSocket` buscam sessão a partir do cookie; frontend não autentica jogador com `playerToken` local.
- `/api/auth/me` renova sessão válida, retornando perfil + stats.
- `/api/auth/logout` apaga sessão do banco e expira cookie.

Risco a registrar: a validade de cookies em vários dispositivos depende das linhas em `user_sessions`; alterar senha por reset remove sessões de todos os dispositivos.

2. Recuperação de senha

Recuperação baseia-se em **nickname e data de nascimento**; `/api/auth/recovery/verify` limita a 8 tentativas a cada 15 minutos por par IP+nickname, gera token de reset aleatório com TTL **10 minutos**, salvo em `Map` no processo. `/api/auth/recovery/reset` verifica token e altera senha. O código não declara envio por e-mail ou SMS. Esse mecanismo tem implicações de segurança: quem conhece nickname e data de nascimento pode tentar recuperação; para produção com ameaça elevada, considerar fator de verificação adicional **como melhoria futura, não funcionalidade já entregue**.

3. Fotos, avatares e privacidade

A conta pode usar um dos 10 animais em `/public/avatars/*.png` ou uma foto personalizada gravada em `BYTEA` em `users.profile_image`. O campo `avatar_type` distingue `preset` e `upload`. Endpoints de perfil autenticados atualizam a escolha; `GET /api/profile/image/:id` serve os bytes com MIME. Fotos e data de nascimento são dados pessoais: **não copiar para anexos públicos, prompts de IA ou dumps não protegidos**. O token usado para reconexão da sala não autentica conta.

4. Salas privadas, host e Socket.IO

Senha de sala é armazenada **somente em memória** via `scryptSync` com salt aleatório; verificação usa `timingSafeEqual`. Há limite para tentativas de ingresso em sala privada. Host é determinado pelo token admitido e pelo estado do socket, não por valor vindo do cliente. Existe separação jogador vs. espectador,

controle de ação por fase e conferência de sessão para reconexão. Código de sala não deve ser tratado como senha: sala privada exige segredo adicional.

5. Backend para backend e gestão de segredos

Variáveis do portal, conforme `.env.example`, documentos e código:

Nome	Uso	Atenção
`PORT`	porta HTTP (padrão 8080)	número válido
`DATABASE_URL`	conexão PostgreSQL portal	segredo crítico
`DB_HOST/DB_PORT/DB_USER/DB_PASSWORD/DB_NAME`	alternativa de conexão	não combinar incorretamente com `DATABASE_URL`
`AUTO_INIT_DB`	habilita bootstrap base	ainda há migrations e importações fora desse flag
`TRIVIA_API_URL`	base do serviço de perguntas	ex.: URL pública do app separado
`TRIVIA_API_KEY`	chave de serviço para a Trivia	**somente backend**
`TMDB_READ_ACCESS_TOKEN`	API de imagens TMDB	segredo
`IGDB_CLIENT_ID`	credencial client de IGDB	segredo/identificador
`IGDB_CLIENT_SECRET`	segredo Twitch/IGDB	segredo
`IGDB_ACCESS_TOKEN`	possível token obtido/ configurado	segredo

Os nomes adicionais/alternativos devem ser validados em `whoami-images.js`; não presumir que todos são obrigatórios simultaneamente. No serviço Trivia: `TRIVIA_API_KEY`, `TRIVIA_ADMIN_KEY`, `DATABASE_URL` ou credenciais DB_* e opções de importação (`AUTO_IMPORT`). Nunca duplicar a `TRIVIA_ADMIN_KEY` no portal; só chave de serviço.

Modelo seguro de configuração

```
# arquivo real protegido e não versionado
PORT=8080
DATABASE_URL=postgresql://USUARIO:SENHA@HOST_PRIVADO:5432/BANCO
TRIVIA_API_URL=https://ronizada-quest.discloud.dev
TRIVIA_API_KEY=<SEGREDO_DIFERENTE_DA_CHAVE_ADMIN>
AUTO_INIT_DB=false
# variáveis opcionais dos provedores, quando efetivamente configuradas
```

Valores acima são PLACEHOLDERS. É proibido publicar ZIP com senhas reais por acidente. Os ZIPs de referência da documentação não exigem embutir `.env`. Idealmente configurar segredos pelo painel/ gerenciador apropriado; se precisar enviar `.env` no deploy privado, manter a cópia fora de repositórios.

6. Inicialização e migrações

`server.js` conecta banco, opcionalmente executa `bootstrapDatabase`, **sempre chama** `runMigrations` e `loadQuizQuestions`, e então inicia servidor. Isso implica testes de startup contra banco **separado**. O backend Trivia executa preparação limitada ao schema `trivia` ao subir, e pode importar em modo `if_empty`. Não automatizar uso de usuário PostgreSQL privilegiado na API de perguntas.

7. Observabilidade realmente presente

Há `console.log` / `console.error` no servidor para conexões, início, erros de sorteio, comunicação e autenticação. **Constatação importante:** o código atual registra em log a palavra e a identidade do impostor no começo de rodada; isso deve ser tratado como risco de exposição de informação de jogo a quem acessa logs e precisa de revisão antes de operar logs públicos ou compartilhados. A API fornece `/health`. **Não se verificou** Prometheus, Grafana, Loki integrado ao site, Sentry, tracing OpenTelemetry, alertas de uptime, métricas HTTP estruturadas ou auditoria de mudanças administrativas nesta implementação. Recomendações de observabilidade devem aparecer como melhorias futuras.

8. Incidentes comuns — diagnóstico sem expor segredos

Sintoma	Causa provável e diagnóstico
`/api/trivia/categories` 503 / UI "API indisponível"	conferir `TRIVIA_API_KEY` no backend do portal, `TRIVIA_API_URL`, conectividade e `/health` do serviço; nunca expor valor da chave no chat
Falha `getaddrinfo ENOTFOUND` para DB privado	verificar VLAN/host privado da Discloud; `localhost` não é DB remoto
Erro `password authentication failed`	role/senha/URL DB divergentes; confirmar usuário restrito e parâmetros no app correto
UI antiga depois de atualização	cache de `style.css?v=...` e Ctrl+F5; conferir arquivo implantado
Sala desaparece após deploy	estado em memória do portal, comportamento esperado nesta arquitetura
Foto de perfil não aparece	sessão, `avatar_type`, bytes no banco, endpoint de imagem/MIME; não copiar só PNG preset
Pergunta expira mas jogador tenta responder	retorno 410 da Trivia; mostrar "Tempo esgotado" sem marcar acerto
Quem Sou Eu? sem imagem	confirmar token do provedor, resolução/fallback e URLs; não quebrar o jogo inteiro
Categoria visível com zero perguntas	categoria filtrada por `available > 0` no portal; revisar API e cache de 60s

9. Rotina de publicação recomendada

1. Identificar app de destino por ID real no painel, não pelo nome do arquivo ZIP.
2. Salvar ZIP anterior e manifesto original da Discloud.
3. Confirmar `.env` não foi sobrescrita, segredo não consta em ZIP público.
4. Testar `npm ci` e `npm test` localmente.
5. Usar banco de homologação para versões beta; **mesmo AUTO_INIT_DB=false pode executar migrações/importações**.
6. Inspecionar `package.json`, scripts de migração, diferenças de SQL e `discloud.config` antes do deploy.
7. Publicar a aplicação correta e verificar inicialização + HTTP + Socket.IO + duas abas/usuários.
8. Validar Impostor, Quem Sou Eu? e Corrida, inclusive rollback.
9. Se houver falha, restaurar apenas a aplicação afetada, sem comandos destrutivos no banco.

10. Riscos técnicos e recomendações (NÃO implantadas automaticamente)

- Persistir salas para suportar restart ou implementar avisos/reconexão robustos.
- Separar worker de migrations do startup e controlar versões no banco.
- Fortalecer recuperação de conta (segredo de recuperação fora da data de nascimento).
- Implementar políticas de CSP, limites de chamadas por API externa e tratamento homogêneo de erros.
- Implementar consentimento/retenção/remoção de dados pessoais, se exigido pela legislação.
- Revisão factual das 5.046 perguntas liberadas em lote.
- Adicionar auditoria automatizada de acessibilidade e monitoramento real.

Esses são **itens de evolução**, não recursos presentes no código-fonte analisado.

---

09 — QA, TESTES DE REGRESSÃO, ACEITAÇÃO E FIDELIDADE

1. Objetivo de qualidade

Uma reconstrução aceitável deve provar **equivalência funcional e visual** nos três jogos e no shell comum. Apenas renderizar a home e a tela da Corrida não basta. O pacote do portal declara `npm test` com runner `node --test tests/*.test.js`; os testes atuais incluem `impostor-vote`, família `whoami-*`, `trivia-race`, `trivia-final-screen`, `trivia-visual-refinement`. Testes da Ronizada Quest têm grupos de normalização, bootstrap e rounds V1.3.

Historicamente a V20.20.4 passou em 60 testes reportados na conversa do projeto. Esse número não substitui rodar a suíte novamente no ambiente de destino nem comprova testes ponta a ponta remotos. Não prometer disponibilidade de APIs externas por testes unitários.

2. Matriz de validação obrigatória

ID	Área	Etapa	Resultado esperado
A01	Shell	visitar `/`	biblioteca sem erros e card de três jogos
A02	Navegação	abrir URLs diretas das três modalidades	mesma SPA com tela correta, sem 404
A03	Perfil	cadastrar conta válida	cookie HttpOnly, perfil exibido com apelido
A04	Perfil	login inválido	401 e feedback, nenhum cookie novo
A05	Avatar	trocar preset e usar foto cadastrada	imagem consistente em lobby e jogo
A06	Sala	criar sala pública; segundo cliente entrar por código	mesmo código, ambos presentes
A07	Privacidade	sala com senha errada/correta	nega/aceita conforme credencial
A08	Host	membro comum tenta iniciar partida	ação negada
A09	Ready	membro não pronto tenta iniciar	host não consegue iniciar
A10	Reconexão	atualizar aba em partida e retomar dentro da tolerância	identidade conservada, estado readquirido
A11	Abandono	host sai	transferência/encerramento conforme código; nenhum cliente preso
I01	Impostor	iniciar com 2 pessoas	impedido (mínimo 3)
I02	Impostor	iniciar com 3, todos prontos	papéis privados e uma palavra
I03	Impostor	votar em si e votar 2×	ambos impedidos
I04	Impostor	empate na votação	desempate sem perder estado
I05	Impostor	encerrar rodada	vencedor/side correto e histórico pertinente
Q01	Quem Sou Eu?	iniciar com 2+	inicia revisão, personagens não duplicados
Q02	Quem Sou Eu?	autoavaliar personagem	proibido

ID	Área	Etapa	Resultado esperado
Q03	Quem Sou Eu?	todos aprovam / pedem reroll	transição/reroll apenas conforme unanimidade
Q04	Quem Sou Eu?	palpite próximo	não gasta tentativa
Q05	Quem Sou Eu?	três erros	elimina e registra tentativas
Q06	Quem Sou Eu?	`allFinish` ativo	encerra apenas ao fim de todos
Q07	Quem Sou Eu?	falha de imagem externa	fallback sem travar a sala
T01	Corrida	GET `/api/trivia/categories`	retorna categorias e defaults sem chave no navegador
T02	Corrida	tags selecionar/remover/limpar/todas	estado visual e payload de IDs canônicos
T03	Corrida	zero categorias	botão criar desabilitado/erro
T04	Corrida	dificuldade aleatória/fixa	filtro respeitado na questão
T05	Corrida	15s padrão, 0 e 30	cronômetro exato e validação 0..30
T06	Corrida	resposta dupla	primeira aceita; segunda 409
T07	Corrida	resposta tardia	API responde 410; jogador +0
T08	Corrida	dois clientes, pergunta única	ambos veem as mesmas opções/roundId
T09	Corrida	todos responderam	encerra imediatamente sem esperar prazo
T10	Corrida	resposta correta	sobe exatamente uma casa
T11	Corrida	erro/ausente	torre não avança
T12	Corrida	rodada resolve	gabarito só após `/finish`, próximo sorteio após 4s
T13	Corrida	empate na chegada a 10	pergunta extra só finalistas, sem pontuar acima de 10
T14	Corrida	vitória única	tela de campeão, foto, ranking, rodadas, ação host
T15	Corrida	API de perguntas falha	erro controlado sem pontos falsos
D01	Visual	telas em 1366×768 e 1536×864	sem botões inacessíveis e alinhamento de controles
D02	Visual	mobile 390×844	conteúdo acessível, sem corte lateral global
D03	Visual	input tempo	valor “15” nítido, − e + funcionam
D04	Visual	select/checkbox	**sem faixas rosas** residuais
D05	Visual	Código/Senha/Entrar	três controles de **52px de altura externa** na Corrida
D06	Visual	font loading	Chakra Petch + Nunito; sem Pixelify desatualizada
D07	Visual	consistência dos jogos	mesma identidade retro, nenhum painel preto exclusivo

3. Testes de segurança/privacidade

- Testar acesso à `/api/v1/*` sem chave → 401; nunca embutir chave no HTML/JS/resposta pública.

- Login com sessão falsificada não pode renomear perfil de outro usuário.
- Um jogador em sala não pode responder pela identidade de outro através de `playerId` forjado.
- Ações de host são negadas para outros sockets.
- Um espectador não pode alterar votos nem respostas.
- Foto de perfil não deve permitir conteúdo/script inseguro; respeitar MIME/limites implementados.
- Senha da sala e segredo de sessão não devem aparecer em logs/JSON público.

4. Testes com conexão real

Usar 2–3 navegadores/abas independentes (perfis de browser separados para evitar compartilhar cookies involuntariamente) e banco de **homologação**. Manter clocks do servidor e do banco coerentes; testar prazo usando registros do PostgreSQL. Validar o modo sem cronômetro separadamente. Atualizações na API podem funcionar enquanto o portal antigo ainda está online; isso não demonstra que a integração do jogo foi validada.

5. Validação visual pixel comparativa

1. Capturar screenshots das mesmas rotas/estados no sistema de referência e na cópia, no mesmo browser, zoom **100%** e viewport fixa.
2. Manter escala, devicePixelRatio, fonte carregada e número de jogadores idênticos.
3. Comparar fundo, grid 42px, bordas de 3px, título, altura barra 42px, sombra 8×9px, conteúdo creme, posição dos botões, tags e labels.
4. Checar 1366×768, 1536×864 e 390×844; adicionalmente testar zoom 125% e foco pelo teclado.
5. Medir e corrigir divergências de CSS; não tentar “consertar” com `overflow:hidden` no body se ocultar ações.
6. Repetir screenshot após mudanças em uma regra global (`.retro-button`, `.retro-input-wrap`, fontes), pois afetam **todos os jogos**.

6. Checklist para liberação

```
[ ] Backup da versão publicada disponível e confirmado
[ ] Arquivos fonte e manifestos revisados
[ ] .env presente APENAS no ambiente privado apropriado
[ ] npm ci + npm test executados sem falhas
[ ] Banco/teste separado utilizado para homologação
[ ] Todos os três jogos iniciam e concluem
[ ] Modo espectador/reconexão/host verificados
[ ] Trivia V1.3 health/categories/answers/finish validados
[ ] Teste de atraso (HTTP 410) e duplicata (409) confirmado
[ ] UI desktop e mobile inspecionadas
[ ] Fontes Chakra Petch/Nunito carregadas
[ ] Entrar + inputs com alturas padronizadas; sem faixas rosas
[ ] Tela de campeão testada
[ ] Produção atualizada somente após testes e rollback definido
```

7. Escopo não testado automaticamente

A presença de teste automatizado não prova que o banco remoto está íntegro, que URLs de terceiros respondem, que todas as imagens estão corretas, nem que hospedagem irá manter Socket.IO estável para muitos jogadores. Medir carga/latência e auditar acessibilidade/segurança são tarefas extras. Este documento distingue testes históricos de evidência atual.

---

10 — PLAYBOOK DE RECONSTRUÇÃO PARA OUTRA INTELIGÊNCIA ARTIFICIAL

1. Missão

Objetivo: reconstruir um portal web quase idêntico ao **Caroninhas Party V20.20.4** em estética, navegação, regras, API, banco e comportamento multiplayer, usando o conjunto de documentos, capturas de referência e, quando autorizado, os dois ZIPs-fonte do projeto. **Não** criar um conceito inspirado no Caroninhas; implementar a mesma solução.

Prompt-mestre pronto para copiar

> Você é uma equipe sênior de frontend, backend Node.js, PostgreSQL, WebSocket, QA e design de interfaces. Sua tarefa é REPRODUZIR o Caroninhas Party conforme os arquivos `00` a `11` desta pasta. Priorize equivalência exata de UX, comportamento e arquitetura. O frontend deve usar HTML5, CSS3 e JavaScript puro, Express/Socket.IO/pg no backend e a API Ronizada Quest separada para perguntas. A interface deve seguir a paleta rosa/creme, grade de 42px, barras retrô, Chakra Petch + Nunito, bordas de 3px e sombras duras. Preserve os três jogos Impostor, Quem Sou Eu? e Corrida do Conhecimento. Reutilize identidade, contas, avatares, salas, lobbies, pronto, host, senha, espectador, reconexão e telas de resultado em todas as modalidades. Represente a Corrida com 24 tags filtráveis, dificuldade random/easy/medium/hard, relógio 0..30, torres 0..10, respostas temporizadas pela API, desempate extra e tela de campeão. Não invente APIs, bibliotecas, telas, coleções ou estados; marque explicitamente qualquer dado não fornecido. Nunca coloque chaves ou credenciais no frontend. Use banco de TESTE, valide os contratos e escreva testes equivalentes antes do deploy. Entregue código modular, esquema de banco, scripts, testes e evidências visuais. Ao concluir, liste divergências conhecidas e requisitos não verificados.

2. Não começar por CSS isolado

Uma boa reconstrução precisa de:

1. **Banco e domínio**: esquema `public`, `trivia`, entidades, credenciais isoladas e bootstrap não destrutivo.
2. **API de perguntas**: rest V1.3 e invariantes de tempo/gabarito/duplicata.
3. **Servidor de salas**: estados, token e segredo de reconexão, host, ready, espectadores, eventos por jogo.
4. **Frontend estrutural**: uma SPA com `screen` e `showScreen`, rotas deep link, perfil, modais e conexão Socket.IO.
5. **Visual**: somente após estrutura pronta, aplicar tokens e cascata de CSS comparando imagens.
6. **Validação**: testar duas conexões reais, fluxo de vitória e rollback.

A ordem reduz risco de criar uma “imagem bonita” sem jogo funcionando, ou refazer game state depois de fazer componentes.

3. Matriz de fidelidade

Item	Fidelidade obrigatória	Tolerância permitida
Lógica das rodadas, voto, tentativas, ponto e desempate	funcionalmente exata	nenhuma alteração voluntária
REST/Socket.IO: nomes de eventos, payloads de uso, autorização	exata	somente adaptação explicitada

Item	Fidelidade obrigatória	Tolerância permitida
Dados e perguntas	depende de acervo real	placeholders apenas em homologação, identificados
Shell visual, cores, tipografia e janela	próxima do pixel	diferenças de renderização de fontes por OS
Avatares preset	mesmos arquivos/ordem	nenhuma troca sem autorização
Fotos de conta	mesma origem (PostgreSQL) e endpoint	conteúdo pessoal não acompanha entrega
Provêdores externos	mesma hierarquia/categoria/fallback	URLs e respostas são dinâmicas
Persistência de sala	efêmera como referência	melhoria estrutural só numa versão nova e aprovada
Responsividade	respeitar telas 390/1366/1536 e zoom	scroll interno controlado quando necessário
Metadados de social preview	OG/Twitter presentes	mídia do banner exato exige asset original

4. Contratos de aceitação da réplica

Comum

- Biblioteca e três cards; navegação sem refresh entre estados.
- Registro/login por nickname; perfil e estatísticas, avatar default/preset/upload.
- Salas de 4 caracteres, pública/privada, senha com hash, host, ready, join por código, listagem e espectadores.
- Reconexão por `playerToken+roomCode+sessionSecret` dentro de 30s de tolerância.

Impostor

- Mínimo 3, papel secreto exclusivo, palavra/dica, discussão/ordem, voto não duplicável, empate/revotação, eliminação e lado vencedor.

Quem Sou Eu?

- Mínimo 2, múltiplos personagens, revisão coletiva, aprovação/reroll, limite 3 erros, quase acerto sem tentativa, allFinish opcional, imagens e fallbacks.

Corrida

- 24 tags disponíveis quando API retorna, 12 defaults canônicos e aliases corretos.
- Dificuldade aleatória/fixa, 0..30 segundos, privacidade, 2+ jogadores, perguntas idênticas por rodada e seleção imutável.
- Relógio validado no PostgreSQL da Trivia, 1 ponto por acerto, sem pontos para atrasado/ausente, limite 10, desempate só finalistas.
- Transição automática de 4s e tela de campeão com ranking e botão de revanche apenas ao host.

Visual

- Rosa quadriculado `#ead7e2`, linhas 42px; janela creme `#f7f1df`; bordas `#2f2927`; barra rosa `#f3a6b9`; sombra dura 8/9px; título `.exe`, três dots.

- Chakra Petch (substituto de Pixelify na V20.20.4) e Nunito; legibilidade de números.
- Campos código/senha + botão Entrar da Corrida com 52px, no mesmo alinhamento.
- Sem faixas verticais rosas decorativas em select/checkbox.
- Mesma identificação de nome/avatar em todas as modalidades.

5. Arquivos prioritários para leitura por agentes

```
1. public/index.html       (rotas de tela, shell, campos, SVG, conteúdo)
2. public/style.css        (valores CSS e regras media da versão)
3. public/script.js        (estados e interação real)
4. server.js               (salas, Socket.IO, Impostor, Quem Sou Eu?)
5. trivia-race.js          (corrida, API e desempate)
6. auth.js                 (conta, cookies, perfil, stats)
7. whoami-images.js        (buscador de imagens)
8. database/schema.sql + database/migrations/* + seed.sql
9. Ronizada Quest src/app.js + src/server.js + sql/001_init.sql
10. tests/ de ambos os projetos
```

Se o agente não receber os ZIPs-fonte, pedir os **arquivos que faltam** e reconstruir as partes especificadas, mas sinalizar aproximação inevitável. **Nenhum documento textual garante reprodução pixel perfeita de 6.000 linhas de CSS nem o acervo completo de milhares de registros sem os assets reais.**

6. Processo de implementação por sprints (sugestão, não estado atual)

Fase	Entregáveis	Condição de avanço
0	Inventário + branch/cópia isolada	testes antigos preservados
1	DB + scripts + serviço de autenticação	cadastro/login/avatares OK
2	serviço de salas/socket/reconexão	duas abas e host/ready OK
3	Impostor	votação/empates/regressão OK
4	Quem Sou Eu?	revisão, palpites, imagens/allFinish OK
5	Trivia API	201/409/410/finish + SQL OK
6	Corrida	filtros/tempo/torres/campeão OK
7	CSS + social + responsive	screenshots e UI idênticos
8	Hardening e deploy de teste	plano de rollback, smoke real

7. Prompts para revisão especializados

Backend

> Compare cada `socket.on` e a transição de fase do código reconstruído com `server.js`, `trivia-race.js` e `03_CONTRATOS_HTTP_E_SOCKET_IO.md`. Liste divergências linha a linha. Procure vazamentos de gabarito e de personagem oculto; rejeite mudanças de fase sem autorização de host.

Frontend

> Compare o DOM de todas as 16 telas, nomes de IDs/classes, modais, armazenamento de token e callbacks com `index.html` e `script.js`. Verifique que mudanças da Corrida não alteram visual/funcionalidade do Impostor e Quem Sou Eu?.

Design

> Renderize 1366×768, 1536×864 e 390×844 nas mesmas condições das referências. Compare grid, cores CSS, fonte Chakra Petch/Nunito, janelas e ações. Meça sobreposição, clipping, scroll e legibilidade do tempo.

Segurança

> Procure segredos em bundle/browser/log e SQL sem parâmetros. Teste sala privada e troca de host, cookie HttpOnly, acesso por token de outro jogador, expiração em banco. Não confunda `playerToken` com sessão de conta.

8. Alterações proibidas sem autorização específica

- Sobrescrever `caroninhas-party` com ZIP de `ronizada-quest` ou `caroninhas-party-v2020-test`.
- Alterar/remover schema `public` para instalar trivia.
- Trocar visual retrô pastel por dark mode, blue SaaS ou glassmorphism.
- Redefinir o modo de vitória ou as pontuações.
- Tirar login/perfil/avatares/Quem Sou Eu?/Impostor por considerá-los “não essenciais à demo”.
- Transferir chave admin da Trivia ou `DATABASE_URL` para browser.
- Inventar que estatísticas de Corrida estão persistidas em `match_history` sem implementação.

9. Entrega requerida do agente construtor

Código-fonte completo; `README` com instruções por ambiente; `package-lock`; migrations/scripts; `.env.example` sem segredo; testes unitários e E2E quando possível; relatório de acessibilidade parcial; screenshots por viewport; tabela de paridade de regras; listagem do que foi aproximado; ZIP de homologação com ID de teste; guia seguro para promoção a produção.

---

11 — INVENTÁRIO GERADO DIRETAMENTE DOS ARQUIVOS-FONTE

> Gerado a partir dos ZIPs específicos da V20.20.4 e da API V1.3; lista de itens observados no código, sem anexar credenciais nem dados pessoais. Códigos-fonte e linhas podem mudar em outros builds.

1. Hash SHA-256 dos arquivos essenciais

Serviço	Arquivo	SHA-256	Linhas
Portal	`server.js`	`e5b234327ff51422551d7e12f27e6e11786548cebbb62e91d116b4fabae854dc`	3109
Portal	`trivia-race.js`	`c26bce42c86f50a65d9c73a17c564828e75978af6c5bbaa85dfd3b3e6a90e93f`	294
Portal	`auth.js`	`5eae2acc6be3ef92626ca62ab6fc2552e1f537c3674da5326a43c94427904d01`	771
Portal	`db.js`	`ae497d100428f2b9b310217054bdcba0babb33a62ca7f8620de5234c8c07b638`	46
Portal	`whoami-images.js`	`5aa4e5fc75f4a526082f676544a5ac3636f04ea189597bbb883786ad55a841f1`	378
Portal	`public/index.html`	`9c3af5383a2227749617f51cd9bbf5842553c7d7f912818408a6b6af318eb689`	1651
Portal	`public/script.js`	`dc91bd922b42be4149e3ec3d02f00ac9905edb2f3c9701436d86e3f644a9033b`	5060
Portal	`public/style.css`	`c70b7f0069205572f19d18d81289e915e072145b7722feb08f7e8492713ded9b`	6182
Portal	`database/schema.sql`	`6202b3818c7d90b6a4215b2d5dbfb1622cfe44a67717db68e94d881584c7f057`	173
Portal	`database/seed.sql`	`b81bb2b80d3b166f3efdf0ac7a3088687f946fe82380eb0dc65284960442aa5e`	654
Portal	`package.json`	`a95a7e4398f75d30403f46198ac0326a3a4ae60a39901a9e793bc9814f332c89`	20
Portal	`discloud.config`	`164023f0dec29faab5291601ec5dea2a8f5b83d33cd1f96e284ca90192a17524`	11
Trivia API	`src/app.js`	`b36ca6b0772250689855769503e70e26d677ece7d0d11b169644524e3a811b17`	229
Trivia API	`src/server.js`	`dbfd6a565203d553f87c11c62f1647732387e08eeed925c7379f82b2a292b9c0`	50
Trivia API	`src/bootstrap.js`	`b1d5ee8a959da2f6160b55682ea10b8aebd9e3b4bd743cb61b6ba35d721ecb0f`	56
Trivia API	`src/db.js`	`69b19dd1fac7b23df9df426a9042fe41db28144bbcb8a914ceff72605f666337`	21
Trivia API	`src/rounds.js`	`724acceb87ab247f217ace3c4a0a35560eed24ac21b8bc4dfa35f6a319952c89`	19
Trivia API	`src/normalize.js`	`1ac1f34851432074221702 4995a7ffb9a7c20425bd2bd9b6812cd451c996a60d`	80
Trivia API	`src/validation.js`	`bbdb4cd39a7245368fdda6e9651f753a20488dd3e0b73b6d1129e95878690229`	29
Trivia API	`sql/001_init.sql`	`9dfc44832a6277d9a04eecc7fe55de6f216de41e22838032978f3ec5a715172d`	55
Trivia API	`package.json`	`1e60fb22602b27092aedaead4fe9064f3654ebb783b287d53c30733a771ae221`	26

Serviço	Arquivo	SHA-256	Linhas
Trivia API	`discloud.config`	`b0f84d4bd7806123ac107c41aa2b4c0d7b75a7f804393d6265f6e236475b6429`	11

2. Roteamento REST detectado por expressões `app.get/post/patch/put`

Portal: server.js

- Linha 32: `app.get(/^\/(impostor|quem-sou-eu|corrida-do-conhecimento)(?:\/[A-Z0-9]{4})?\/?$/i,`
- Linha 37: `app.get(/^\/quiz-e-caos(?:\/[A-Z0-9]{4})?\/?$/i, (req, res) => res.redirect(302, "/"));`
- Linha 1573: `app.get("/api/game-config", async (req, res) => {`
- Linha 1608: `app.get("/api/whoami-config", async (req, res) => {`
- Linha 1645: `app.get("/api/whoami-image", async (req, res) => {`
- Linha 1683: `app.get("/api/rooms", (req, res) => {`

Portal: auth.js

- Linha 335: `app.get("/api/auth/me", async (req, res) => {`
- Linha 349: `app.post("/api/auth/register", async (req, res) => {`
- Linha 380: `app.post("/api/auth/login", async (req, res) => {`
- Linha 411: `app.post("/api/auth/logout", async (req, res) => {`
- Linha 422: `app.post("/api/auth/recovery/verify", async (req, res) => {`
- Linha 460: `app.post("/api/auth/recovery/reset", async (req, res) => {`
- Linha 487: `app.patch("/api/profile/nickname", async (req, res) => {`
- Linha 515: `app.patch("/api/profile/password", async (req, res) => {`
- Linha 540: `app.patch("/api/profile/avatar", async (req, res) => {`
- Linha 568: `app.post("/api/profile/photo", async (req, res) => {`
- Linha 604: `app.get("/api/profile/image/:id", async (req, res) => {`
- Linha 646: `app.get("/api/game-preferences/:gameType", async (req, res) => {`
- Linha 662: `app.put("/api/game-preferences/:gameType", async (req, res) => {`
- Linha 683: `app.get("/api/profile/stats", async (req, res) => {`

Portal: trivia-race.js

- Linha 65: `app.get('/api/trivia/categories', async (req,res) => {`

Trivia: src/app.js

- Linha 25: `app.get('/', (req, res) => res.json({ service:'Caroninhas Trivia API', status:'online', version:'1.3.0', docs:'Consulte o README do pacote; acesso à API requer chave privada.' }));`
- Linha 27: `app.get('/health', async (req, res, next) => {`
- Linha 32: `app.get('/api/v1/stats', service, async (req, res) => {`
- Linha 39: `app.get('/api/v1/categories', service, async (req, res) => {`
- Linha 46: `app.post('/api/v1/rounds', service, async (req, res) => {`
- Linha 93: `app.get('/api/v1/rounds/:id', service, async (req, res) => {`
- Linha 115: `app.post('/api/v1/rounds/:id/answers', service, async (req, res) => {`
- Linha 153: `app.post('/api/v1/rounds/:id/finish', service, async (req, res) => {`
- Linha 180: `app.get('/api/v1/admin/questions', admin, async (req, res) => {`
- Linha 192: `app.patch('/api/v1/admin/questions/:id/status', admin, async (req, res) => {`

3. Eventos Socket.IO recebidos e emitidos (ocorrências detectadas)

server.js

- L200: `io.emit("rooms-list", roomsList());`
- L207: `io.to(code).emit("room-updated", publicRoom(room));`
- L248: `io.to(spectatorChannel(room.code)).emit("spectator-state", buildSpectatorState(room));`
- L523: `io.to(room.code).emit("impostor-vote-finished", buildImpostorFinishedPayload(room));`
- L557: `io.to(room.code).emit("impostor-vote-started", buildImpostorVotePayload(room));`
- L572: `io.to(room.code).emit("impostor-vote-progress", {`
- L609: `io.to(room.code).emit("impostor-tiebreak-started", {`
- L753: `io.to(room.code).emit("host-transferred", {`
- L1090: `io.to(player.socketId).emit(eventName, {`
- L1146: `io.to(room.code).emit("whoami-guess-announced", event);`
- L1147: `io.to(spectatorChannel(room.code)).emit("whoami-guess-announced", event);`
- L1159: `io.to(room.code).emit("whoami-game-finished", buildWhoAmIFinishedPayload(room));`
- L1313: `io.to(room.code).emit("whoami-status-updated", {`
- L1335: `io.to(player.socketId).emit(`
- L1405: `io.to(player.socketId).emit(`
- L1425: `io.to(player.socketId).emit(`
- L1446: `io.to(player.socketId).emit("game-started", {`
- L1452: `io.to(player.socketId).emit("game-started", {`
- L1460: `io.to(player.socketId).emit("impostor-vote-started", {`
- L1471: `io.to(player.socketId).emit("impostor-tiebreak-started", {`
- L1479: `io.to(player.socketId).emit(`
- L1540: `io.to(spectatorChannel(room.code)).emit("room-closed", {`
- L1544: `io.to(room.code).emit("room-closed", {`
- L1695: `io.on("connection", (socket) => {`
- L1701: `socket.on("resume-session", (`
- L1743: `socket.on("abandon-session", (`
- L1767: `socket.on("room-preview", ({ code, gameType }, callback) => {`
- L1775: `socket.on("spectator-get-state", ({ code }, callback) => {`
- L1781: `socket.on("get-rooms", (callback) => {`
- L1790: `socket.on("create-room", async (`
- L1885: `socket.on("create-whoami-room", async (`
- L1981: `socket.on("join-room", async (`
- L2146: `socket.on("set-ready", (`
- L2193: `socket.on("start-game", async (`
- L2316: `socket.on("start-impostor-vote", (`
- L2359: `socket.on("cast-impostor-vote", (`
- L2425: `socket.on("start-whoami-game", async (`
- L2562: `socket.on("whoami-review-vote", async (`
- L2671: `io.to(room.code).emit(`
- L2697: `socket.on("whoami-reveal-self", (`

- L2764: `socket.on("whoami-guess", (`
- L2833: `io.to(room.code).emit("whoami-game-finished", buildWhoAmIFinishedPayload(room));`
- L2887: `socket.on("new-whoami-round", (`
- L2927: `io.to(roomCode).emit("whoami-back-to-lobby");`
- L2933: `socket.on("new-round", (`
- L2973: `io.to(roomCode).emit("back-to-lobby");`
- L2979: `socket.on("close-room", (`
- L3014: `socket.on("leave-room", (callback) => {`
- L3022: `socket.on("disconnect", () => {`

trivia-race.js

- L100: `io.to(room.code).emit('trivia-state',neutral);`
- L101: `room.players.forEach(p=>{ if(p.socketId) io.to(p.socketId).emit('trivia-my-state', {answered:Boolean(state.answers[p.token]),roundId:state.question?.roundId}); }`
- L156: `io.to(room.code).emit('trivia-error',{error:state.error});broadcast(room);`
- L188: `io.to(room.code).emit('trivia-error',{error:state.error});broadcast(room);`
- L209: `io.to(player.socketId).emit('trivia-state',payload);`
- L212: `socket.on('create-trivia-room', async (request={},callback=()=>{}) => {`
- L236: `socket.on('start-trivia-game',async ({code}={},callback=()=>{}) => {`
- L250: `socket.on('restart-trivia-game',({code}={},callback=()=>{})=>{`
- L259: `io.to(room.code).emit('trivia-back-to-lobby');updateRoom(room.code);`
- L262: `socket.on('trivia-answer',async ({code,roundId,optionId}={},callback=()=>{}) => {`

4. Todas as seções de tela de `index.html`

Linha	ID
99	`menu-screen`
175	`impostor-home-screen`
340	`lobby-screen`
428	`game-screen`
522	`impostor-vote-screen`
560	`impostor-result-screen`
629	`whoami-home-screen`
803	`whoami-lobby-screen`
896	`whoami-review-screen`
973	`whoami-game-screen`
1111	`whoami-winner-screen`
1179	`trivia-home-screen`
1228	`trivia-lobby-screen`
1236	`trivia-game-screen`

Linha	ID
1250	`trivia-winner-screen`
1554	`spectator-screen`

5. Categorias no dataset de origem da Trivia (não refletindo liberação em banco)

O JSONL fonte contém 5.247 registros. Os status e quantidades desta seção representam o **arquivo de importação**, não o banco depois da aprovação em lote: a importação tem status de revisão próprios e a base viva pode divergir.

Categoria canônica (`category_pt`)	Quantidade no JSONL
Animais	98
Anime e mangá	203
Arte	58
Celebridades	52
Cinema	300
Ciência e natureza	296
Computação	189
Conhecimentos gerais	465
Desenhos e animações	108
Esportes	175
Geografia	381
História	410
Jogos de tabuleiro	78
Livros e literatura	119
Matemática	77
Mitologia	70
Musicais e teatro	36
Música	488
Política	74
Quadrinhos	78
Séries e televisão	195
Tecnologia e gadgets	39
Veículos	86
Videogames	1172

Total de categorias: **24**; registros: **5247**. Status no arquivo fonte: `blocked`=106, `pending`=5046, `rejected`=95.

6. Avatares preset e mídias de interface

- `public/avatars/bear.png`
- `public/avatars/cat.png`
- `public/avatars/dog.png`
- `public/avatars/fox.png`
- `public/avatars/frog.png`
- `public/avatars/owl.png`
- `public/avatars/panda.png`
- `public/avatars/penguin.png`
- `public/avatars/pig.png`
- `public/avatars/tiger.png`
- `public/caroninhas-party-banner.png` (social preview do site).

7. Inventário de testes

Portal

- `tests/impostor-vote.test.js`
- `tests/trivia-final-screen.test.js`
- `tests/trivia-race.test.js`
- `tests/trivia-visual-refinement.test.js`
- `tests/whoami-all-finish.test.js`
- `tests/whoami-expansion.test.js`
- `tests/whoami-handlers.test.js`
- `tests/whoami-igdb-token.test.js`
- `tests/whoami-image-improvements.test.js`
- `tests/whoami-images.test.js`

Trivia

- `tests/bootstrap.test.js`
- `tests/normalize.test.js`
- `tests/rounds-v13.test.js`

8. CSS: tokens globais declarados no início do arquivo

```css
:root {
  --bg: #ead7e2;
  --grid: rgba(116, 84, 92, 0.10);

  --cream: #f7f1df;
  --cream-2: #fff9e9;
  --cream-3: #eadfbe;

  --ink: #2e2524;
  --muted: #786a65;
  --line: #2f2927;
```

```
  --pink: #f3a6b9;
  --pink-deep: #df7f9a;
  --orange: #ef9c63;
  --lime: #d8e87e;
  --yellow: #f2d36f;
  --mint: #91c9b6;

  --shadow: #2e2524;
  --soft-shadow: rgba(46, 37, 36, 0.15);
}
```

9. Arquivos do projeto portal (sem datasets binários e segredos)

```
.discloudignore
.env.example
CORRECOES_V20_20_2.txt
LEIA-ME_AJUSTE_CREDITOS.md
LEIA-ME_CORRECAO_20_20_2.md
LEIA-ME_V20_12.md
LEIA-ME_V20_13.md
LEIA-ME_V20_14.md
LEIA-ME_V20_14_1.md
LEIA-ME_V20_15.md
LEIA-ME_V20_16_EXPANSAO.md
LEIA-ME_V20_17_IMAGENS.md
LEIA-ME_V20_18_BUSCADOR.md
LEIA-ME_V20_19_TOKEN_E_CREDITOS.md
LEIA-ME_V20_20_3_CAMPEAO_TEMPO.md
LEIA-ME_V20_20_4_REFINAMENTO_VISUAL.md
LEIA-ME_V20_20_CORRIDA.md
auth.js
database/data/quiz_questions.jsonl
database/migrations/001_v19_accounts.sql
database/migrations/002_quiz_caos.sql
database/migrations/003_quiz_pawns.sql
database/migrations/004_game_preferences.sql
database/migrations/005_whoami_expansion_1008.sql
database/schema.sql
database/seed.sql
db.js
discloud.config
package-lock.json
package.json
public/avatars/bear.png
public/avatars/cat.png
public/avatars/dog.png
public/avatars/fox.png
public/avatars/frog.png
public/avatars/owl.png
public/avatars/panda.png
public/avatars/penguin.png
public/avatars/pig.png
public/avatars/tiger.png
public/index.html
public/script.js
public/style.css
scripts/bootstrap-db.js
scripts/init-db.js
scripts/load-quiz-questions.js
scripts/run-migrations.js
scripts/whoami-image-report.js
server.js
tests/impostor-vote.test.js
tests/trivia-final-screen.test.js
tests/trivia-race.test.js
tests/trivia-visual-refinement.test.js
tests/whoami-all-finish.test.js
tests/whoami-expansion.test.js
tests/whoami-handlers.test.js
tests/whoami-igdb-token.test.js
```

```
tests/whoami-image-improvements.test.js
tests/whoami-images.test.js
trivia-race.js
whoami-images.js
```

10. Arquivos da API (sem questões individuais e segredos)

```
.discloudignore
.env.example
.gitignore
CONFIGURAR_BANCO.md
INTEGRACAO_CARONINHAS_V20_20.md
README.md
TESTAR_V1_3.md
discloud.config
index.js
package.json
scripts/bootstrap.js
scripts/import.js
scripts/migrate.js
sql/000_PREPARAR_SCHEMA_E_USUARIO.sql
sql/001_init.sql
src/app.js
src/bootstrap.js
src/config.js
src/db.js
src/importer.js
src/normalize.js
src/rounds.js
src/server.js
src/validation.js
tests/bootstrap.test.js
tests/normalize.test.js
tests/rounds-v13.test.js
```

11. Notas de rastreabilidade

- O inventário é um **índice técnico**; para comportamento exato, abrir o trecho correspondente no código fonte referenciado.
- Nomes de rotas em `RegExp` não são simples literais de string; a seção 2 mostra a expressão de montagem.
- Os valores de secrets não aparecem aqui. Os hashes representam apenas arquivos não confidenciais da cópia de referência.
- Não publicar dump de banco ou fotos de perfil junto com o documento.

---

15 — ATLAS VISUAL DE REFERÊNCIA

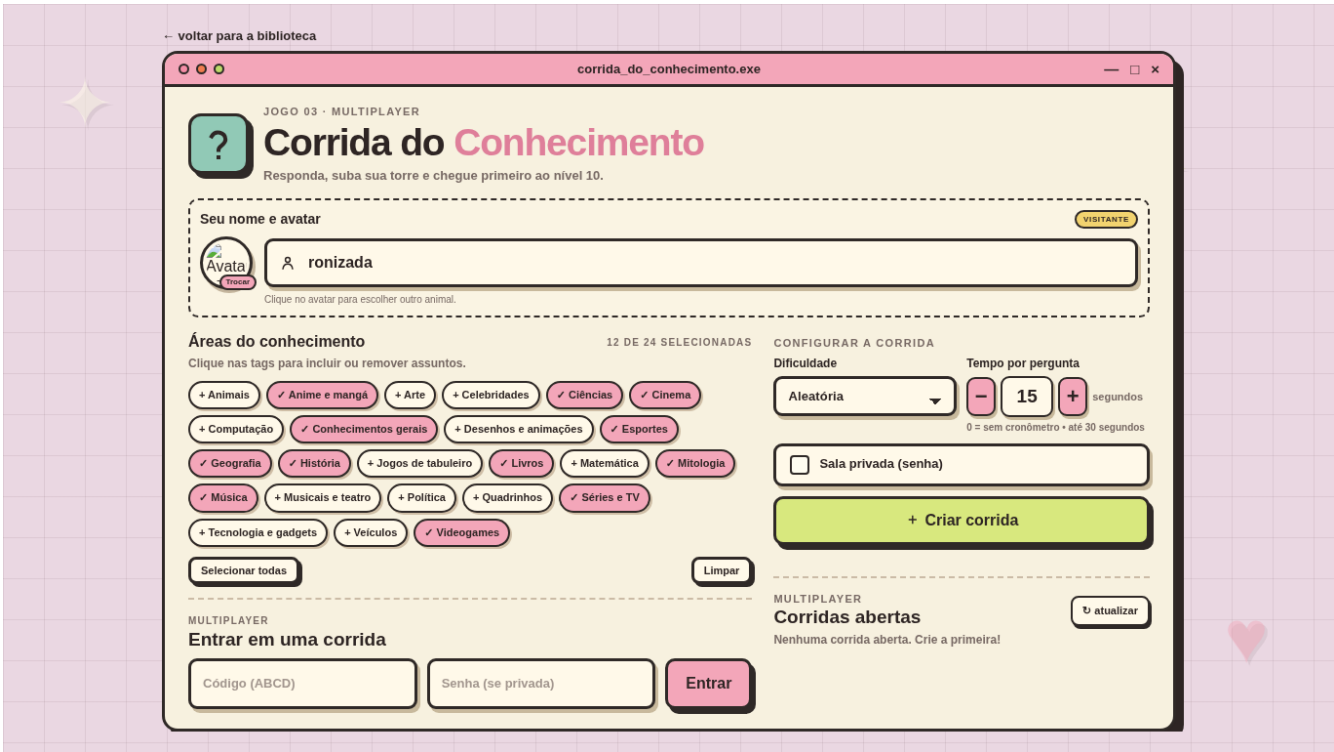
As figuras desta seção são complementares às regras CSS reais. Elas apresentam **capturas de versões/estados diferentes** para que a IA não confunda um erro histórico com o design pretendido. A referência principal para implementação é **V20.20.4** e seus arquivos `public/index.html` e `public/style.css`.

1. Tela de configuração em desenvolvimento — problema histórico



*Esta captura registra problemas do layout anterior (incluindo corte/rolagem) e **não** deve ser copiada como alvo visual final. É evidência do que foi corrigido.*

2. Layout de criação V20.20.4 — prévia em 1366 px



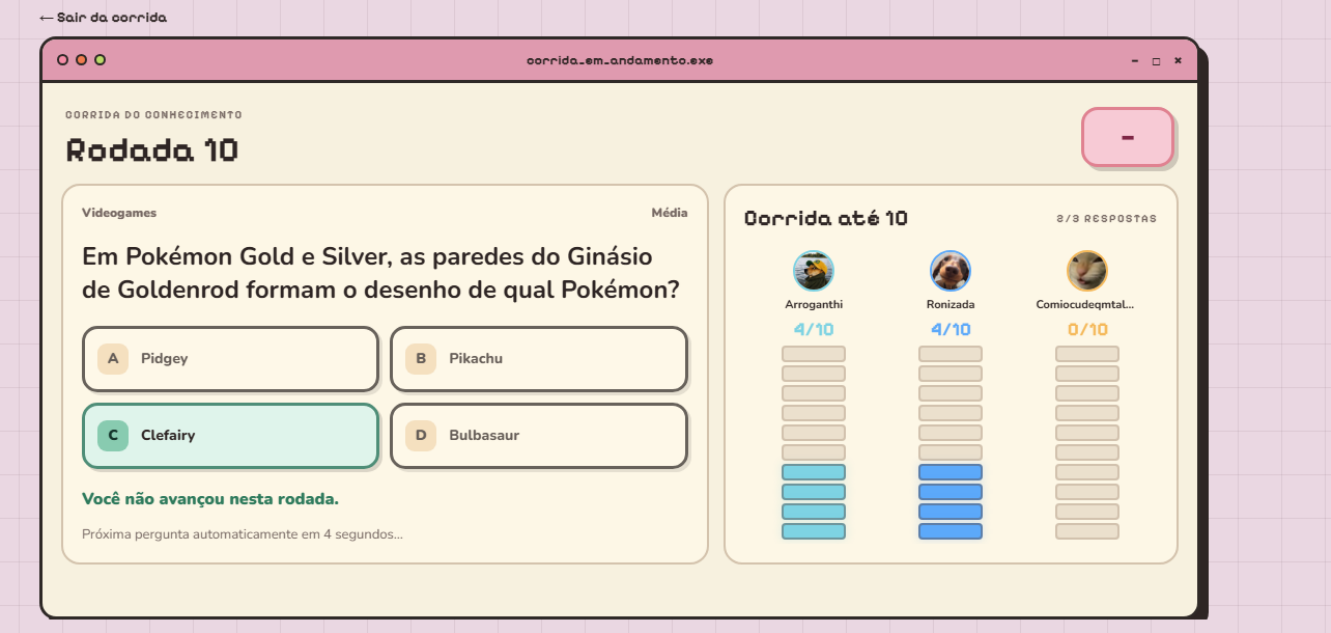
Prévia renderizada em desenvolvimento, não uma comprovação da implementação atualmente publicada. Ela serve como orientação do grid e das proporções; preservar o código CSS efetivo.

3. Layout de criação V20.20.4 — prévia móvel



Exemplo de reorganização responsiva; conteúdo não pode ficar cortado ou inacessível por causa de overflow.

4. Corrida em andamento — exemplo de gameplay real enviado pelo usuário



Referência da composição: pergunta/alternativas à esquerda, três torres com avatar/nome/placar à direita, rosa/creme e bordo escuro. A captura antecede alguns refinamentos da V20.20.4; não serve para reverter suas mudanças tipográficas.

5. Vitória ainda exibida no jogo — problema histórico corrigido



Exemplo do estado anterior à V20.20.3, em que a vitória aparecia dentro da partida. A versão desejada contém uma tela final separada (`trivia-winner-screen`) com campeão, avatar, coroa, classificação e revanche.

6. Hierarquia visual esperada do produto

1. Fundo decorativo quadriculado e elementos lúdicos fixos.
2. Navegação ou botão voltar para biblioteca.
3. Moldura `.retro-window` e barra `*.exe` consistente.
4. Título do modo e conteúdo específico com recortes coerentes.
5. Componentes de nome/avatar e formulários reaproveitados.
6. Informação da rodada e estado de resposta sempre visíveis.
7. Vitória como evento destacado, em tela própria.

Advertência de comparação: captura real, mockup e screenshot de desenvolvimento não têm o mesmo grau de autoridade. A réplica deve ser comparada com o sistema efetivamente publicado na versão-alvo, em mesma viewport e zoom.